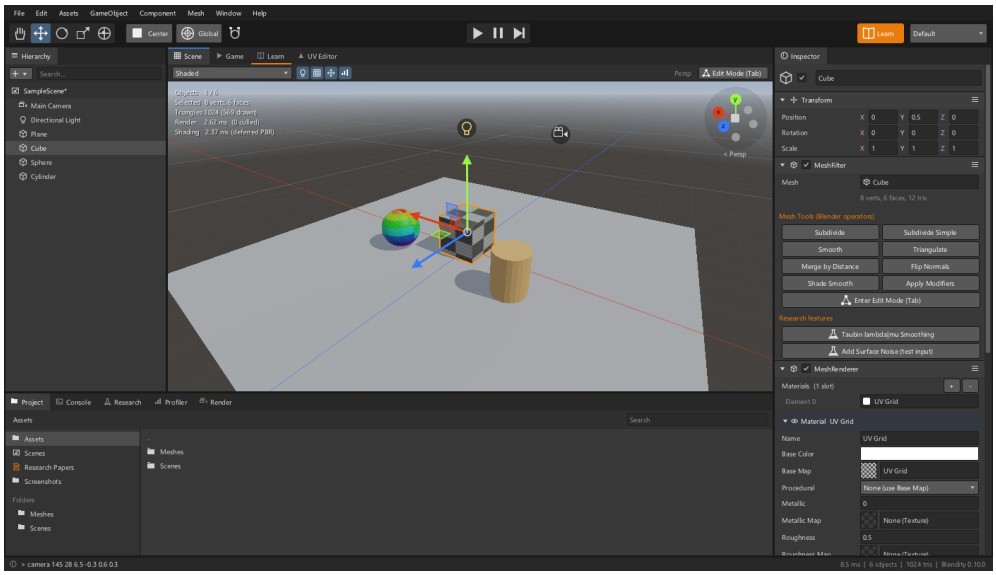
Blendity

Architecture and Engineering

How the editor is built, how it is developed with an AI under human review, and how it is tested and released



Emiliano Manalo, with Anthropic’s Claude

Version 0.13.0 November 2026

Source: GPL-2.0-or-later. Binaries: GPL-3.0-or-later. https://github.com/EmilianoManaloIV/blendity

About this book

This book is for someone who wants to understand how Blendity works well enough to change it: a contributor, a reviewer, a student of how an editor of this size is put together, or a maintainer years from now. It describes the program as it stands at version 0.13.0 and the way it was made, with every borrowed idea, library and paper credited in the References chapter and three indexes that tie the code, the user interface and the documents to those references.

It is written once, as the user asked, rather than re-edited every round; the living documents it condenses (`docs/ARCHITECTURE.md`, `docs/decisions/`, `docs/tasks/`, `CHANGELOG.md`) stay the record of what changed since. Where this book and those files disagree, the files are newer.

The companion volume, *Blendity: The User Guide*, is for the artist: how to navigate the editor and how to feel at home in it coming from Unity, Blender, Maya, 3ds Max, SketchUp, Unreal or Adobe's tools.

Contents

About this book **ii**

1 Purpose and scope **1**

1.1 What Blendity is . . . 1
1.2 Who it is for . . . 1
1.3 What this book covers . . . 1
1.4 Conventions . . . 2
1.5 The numbers at 0.13.0 . . . 2

2 Context **3**

2.1 The people . . . 3
2.2 The operating system . . . 3
2.3 The GPU . . . 3
2.4 Blender’s libraries, optional . . . 4
2.5 Files and other programs . . . 4
2.6 The network . . . 4
2.7 The programs it follows . . . 4

3 Architecture **5**

3.1 Layers . . . 5
3.2 One frame . . . 6
3.3 One edit . . . 7
3.4 Invariants . . . 7
3.5 Module map . . . 8
3.6 Key conventions . . . 9

4 The scene, meshes, undo and assets **10**

4.1 The mesh . . . 10
4.2 The operators . . . 10
4.3 Objects and components . . . 10
4.4 The Reflector . . . 11
4.5 Scene IO and assets . . . 11
4.6 Undo . . . 11

5 Rendering **12**

5.1 The rasterizer . . . 12
5.2 The path tracer . . . 12

5.3 The GPU . . . 13
5.4 Colour management . . . 13
5.5 Lighting . . . 13
5.6 Camera filters . . . 14

6 The editor **15**
6.1 The toolkit . . . 15
6.2 Docking, windows and the module registry . . . 15
6.3 Keymaps, feels and the modal pattern . . . 15
6.4 Tools . . . 16
6.5 The console and headless runs . . . 16
6.6 The Learn tab . . . 16
6.7 Where the HCI work lives . . . 16

7 Bridges, Live Link and file formats **17**
7.1 Live Link . . . 17
7.2 The bridges . . . 18
7.3 File formats . . . 18
7.4 Unity’s project files . . . 18
7.5 Godot, Blender, Unreal . . . 18
7.6 MoonRay . . . 19

8 Research features **20**
8.1 The research folder . . . 20
8.2 What the papers became . . . 20

9 The software development lifecycle **22**
9.1 The roles . . . 22
9.2 The lifecycle . . . 23
9.3 Traceability . . . 24
9.4 Engineering standards . . . 24
9.5 The HCI method in the lifecycle . . . 25
9.6 AI disclosure . . . 25

10 Build, test and release **26**
10.1 Building . . . 26
10.2 The test suites . . . 26
10.3 The audits . . . 26
10.4 Continuous integration . . . 27
10.5 Packaging and deployment . . . 27
10.6 Releasing . . . 28

11 Performance and the minimum machine **29**
11.1 The budget . . . 29
11.2 The minimum machine . . . 29
11.3 What the passes found . . . 29
11.4 Profiling . . . 30

12 What is not recreated, and known limits **31**
12.1 Not recreated from Blender . . . 31
12.2 Not recreated from Unity . . . 31
12.3 Known limits worth knowing . . . 31

13 Licensing and credit **33**
13.1 The terms . . . 33
13.2 Why these terms are compatible with Blender's . . . 33
13.3 Compatible with an AI-assisted project . . . 34
13.4 Due credit . . . 34

References **36**

Index A: the code and its references **48**

Index B: the documents and their references **53**

Colophon **56**

List of Figures

2.1 The system context. Solid arrows are data both ways or out; dashed arrows are optional inputs Blendity runs without. 3

3.1 The layers of `src/`. Each depends only on what is below it; the side modules are small and sit beside the stack. 5

3.2 One edit, from a tool to an undo step. Solid arrows are the call path; dashed arrows are what the edit's version bump reaches. 7

5.1 The two render paths and where they meet. Every GPU path mirrors a CPU path with the same formulas, and the CPU path is the reference the tests compare it with. 12

7.1 The Live Link sequence (`docs/LIVELINK.md`, ADR 0055, ADR 0086). One way out: nothing a client sends is read but its hello. 17

9.1 One task from the request to the release. Diamonds are the human's gates; the boxes with paths are the artifacts. 23

9.2 From a requirement ID to the tests and the audits that hold it. 24

10.1 From a tag to the installed program and its updater. 27

Chapter 1

Purpose and scope

1.1 What Blendity is

Blendity is a 3D editor written in C++17. Its windows, navigation, shortcuts, gizmos, Inspector and Play mode follow the Unity editor [150]; underneath, its meshes, Edit Mode tools, UV unwrapping, Principled materials, render engines (an EEVEE-like rasterizer and a Cycles-like path tracer on the CPU and any Vulkan GPU), skins and IK follow Blender's design and source code [87]. It is not a fork: each subsystem is re-implemented, following Blender's design, so the editor builds and runs with nothing but a compiler, and the matching Blender source file is listed for every module. When Blender's prebuilt libraries are present it links the same ones Blender uses, each behind a switch with Blendity's own code as the fallback (ADR 0001).

It is one process with no plugins and no scripting runtime. Three executables share one static library built from `src/`: `Blendity` (the editor), `blendity_tests` and `blendity_stress`. Everything a tool does is reachable from the Console's command line, which is how the tests, the headless mode and the HCI harnesses drive it.

1.2 Who it is for

The requirements (`docs/requirements.md`) name the audience by what they must be able to do: a Unity-style editor (R-01) with Blender-equivalent modeling (R-02) and SketchUp / UModeler / Plasticity-style direct modeling (R-03), UVs, materials and textures (R-04), rendering with both engines (R-05), file formats matching Blender 5.2 (R-06), every tool reachable from menus, panels, rebindable shortcuts and the console (R-07), adjustable operations (R-08), a Learn tab (R-09), camera filters (R-10), Unity-like global illumination (R-11, R-12), physics for animation as in Cascadeur (R-13), fingers and a pen on a 2-in-1 (R-14), and drawing in perspective with the model built from construction lines (R-15). The HCI reports in `docs/hci/` measure the editor with artists from Unity, Unreal and Adobe's tools in mind, on keyboard, pen and tablet; the later rounds added the VRChat avatar and world pipelines, with the editor reading and writing Unity's own project files.

1.3 What this book covers

Chapters 2 to 7 describe the system: its context, the layers and the two flows that organise everything (one frame, one edit), the scene and mesh model, rendering, the editor toolkit, and the bridges to other programs. Chapter 8 lists the research features and where each paper landed. Chapters 9 to 11 describe how the program is made: the agentic software development lifecycle with its artifacts and gates, the build, test and release machinery, and the performance work. Chapter 12 says what is

not recreated. Chapter 13 states the licence and the credits. The References chapter and the two indexes close the book.

1.4 Conventions

`Monospace` is a file, a function, a flag or a console command; paths are relative to the `blendity/` folder unless they start with `blender/`, which is Blender's source tree beside the project. ADR NNNN is a decision record in `docs/decisions/`, task NNNN a specification in `docs/tasks/`, and round NNN an entry of `CHANGELOG.md` (one entry per round of requests). Numbers quoted here were measured on the machines the CHANGELOG names; where a figure is an estimate, the text says so.

1.5 The numbers at 0.13.0

Source files under `src/` (C++, headers, Objective-C++, CUDA)	380
Lines in `src/`	about 297,000
Lines of unit tests (`tests/`)	about 73,600
Unit checks / tests, the build with Blender's libraries	15,902 / 1,543
Unit checks, the dependency-free build (Windows and Linux)	15,253 / 15,259
Test groups (`--group`)	16
Decision records	104
Task specifications	379
Rounds in the CHANGELOG	409
Learn lessons	45
Windows (module registry)	26
Keymap presets / editor feels	7 / 4
Commits on `main` since 2026-10-06	436

Chapter 2

Context

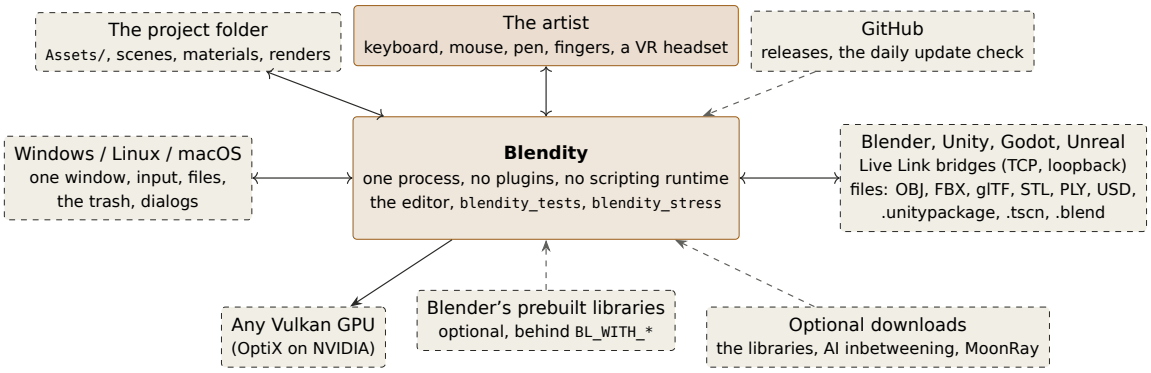


Figure 2.1: The system context. Solid arrows are data both ways or out; dashed arrows are optional inputs Blendity runs without.

2.1 The people

One artist at a time, at a desk or on a 13-inch 2-in-1, with a keyboard and a mouse, a pen, fingers on the screen, a pen display with a dial and express keys, or an OpenXR headset. The HCI work (ADR 0085) names the artists it designs for: people arriving from Unity, Unreal, Blender and Adobe's tools, keyboard-first and pen-first. The one developer is the user, working with an AI (chapter 9).

2.2 The operating system

`src/platform` is one window and its input on Win32, X11 (with XInput 2 for pens and touch) and Cocoa, plus the clipboard, native file dialogs, the trash and DPI, mirroring Blender's GHOST layer [87]. Everything above it is portable C++17 built by `build.bat` on Windows (MSVC, found automatically) and `build.sh` on Linux and macOS, with CMake optional (C-01). Pixels are `0xAARRGGBB` so all three can blit without conversion; the editor redraws only on input, animation or timers and otherwise sleeps in the OS event wait.

2.3 The GPU

Any Vulkan 1.2 GPU: the loader is opened at run time (`VK_NO_PROTOTYPES`), so a machine without one runs on the CPU. The GPU does three things, each with a CPU equivalent: the views are drawn by `gpu::Raster` with the CPU's formulas in GLSL (ADR 0061); the path tracer runs its kernel as a Vulkan compute shader (hardware ray queries where they exist) or, on NVIDIA cards, through OptiX (ADR 0037, ADR 0054); and

lighting bakes gather on it. CPU and GPUs render together, claiming samples from one counter.

2.4 Blender's libraries, optional

When `blender/lib/<platform>` exists beside the project, the build links Blender's own prebuilt libraries (Embree, OpenImageDenoise, oneTBB, Eigen, OpenSubdiv, OpenEXR, OpenColorIO, Manifold, meshoptimizer, Open PGL, Jolt, Vulkan + shaderc, FFmpeg and others), each behind `BL_WITH_<NAME>` with a fallback (ADR 0001, C-02). `BLENDITY_NO_LIBS=1` forces the dependency-free build, which CI tests on every system; the installers ship both programs and Blendity hands over to Blendity-full when the libraries load (ADR 0033). The Console's `libs` prints what a build has.

2.5 Files and other programs

A project is a folder with `Assets/` inside, as Unity's (`BlendityProject` in the home folder unless another is opened). Scenes are text (`.scene`, Zstandard-compressed when the library is there), materials `.mat`, procedural textures `.ptex`, clips `.anim`, sequences `.seq`, layouts `.blendity-ui`. Models move through OBJ, FBX, glTF, STL, PLY and USD, verified both ways against Blender 5.2 (R-06), and through Unity's `.unitypackage`, `.prefab`, `.unity`, `.mat` and `.anim`, Godot's `.tscn` and Blender's `.blend` (through the installed Blender). Live Link streams the scene to Blender, Unity, Godot and Unreal on the same computer as it changes (chapter 7).

2.6 The network

Two uses, both optional and both the user's choice: a once-a-day check of GitHub's releases for a newer build (`src/core/releases.cpp`; Preferences > Updates), and Live Link's TCP server on 127.0.0.1 (no other computer can connect; it is one way out). There is no telemetry and no cloud; the only learned model in the product is the optional, local AI inbetweening download (ADR 0046).

2.7 The programs it follows

The context has a second ring: the programs whose ideas the editor reproduces, each credited in chapter 13 and `CREDITS.md`. Unity for the editor's shape, Blender for the data and the tools, SketchUp, UModeler and Plasticity for direct modeling, ZBrush and Sculptris for sculpting, Substance Painter for texturing, SketchBook, Photoshop and Krita for drawing, Mixamo and the CATS plugin for rigging and avatars, Cascadeur for physics in animation, Premiere for the sequencer, Lightroom for the Editing tab, Unity's Mecanim for retargeting. Where Blendity differs from them, the user guide says so at the moment it matters.

Chapter 3

Architecture

3.1 Layers

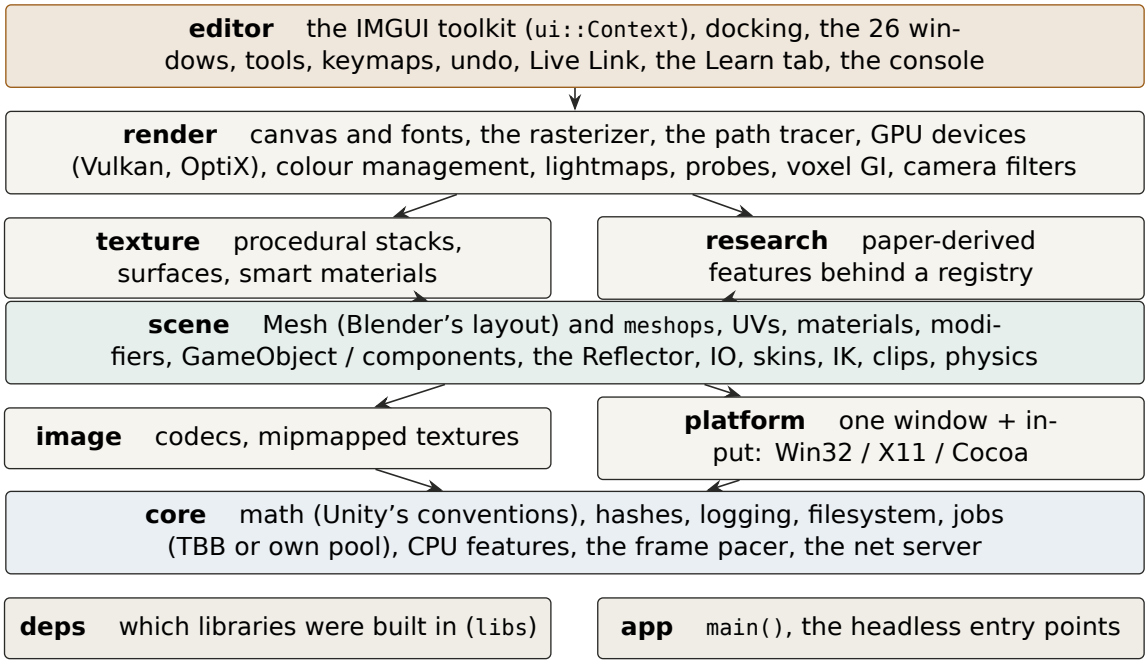


Figure 3.1: The layers of `src/`. Each depends only on what is below it; the side modules are small and sit beside the stack.

Layer	Owns	Depends on
`core`	math (Unity's left-handed, Y-up conventions), hashes (FNV-1a and the per-frame mixer), logging, the filesystem, jobs (TBB or its own pool), CPU features, the frame pacer, the TCP server	–
`platform`	one window, input events, clipboard, file dialogs, the trash (Win32 / X11 / Cocoa)	core
`image`	PNG, JPEG, TGA, BMP, HDR decoding (EXR with OpenEXR), JPEG / HDR / PNG encoding, mipmapped textures	core
`scene`	`Mesh` (Blender's corner-based layout) and the `meshops` operators, UVs, materials, modifiers, `GameObject` / components, scene and model IO, skins, IK, animation clips, sequences, physics, sketch data	core, image
`texture`	procedural texture stacks, surfaces (the layer stack on an object), smart materials	scene
`render`	2D canvas and fonts, the rasterizer, the path tracer, GPU devices, colour management, lightmaps, probes, voxel GI, mesh maps, camera filters, video encoding	core, image, scene
`editor`	the IMGUI toolkit (`ui::Context`), docking, the windows, tools (draw, knife, push/pull, modal transforms, sculpt, paint, sketch), keymaps, undo, Live Link, the Learn tab, the console	everything above
`research`	paper-derived features behind a registry (cited in Learn > References)	scene
`deps`, `app`	which libraries were built in (the `libs` command); `main()` and the headless entry points	all

`scene` never includes `editor` or `render`. Mesh operators (`meshops::*`, in `src/scene/mesh_tools*.cpp`) are pure functions on a `Mesh` plus selection vectors, which is what lets the tests and the stress suite call them directly, and what keeps the editor a thin layer of intent over the data.

3.2 One frame

`platform` events → `Editor::frame` → the windows draw and handle input in docking order (the Scene view runs the active tool: `draw_update`, `pushpull_update`, `modal_update`, the gizmo) → `handle_shortcuts` (the keymap; tools consume the keys they own first) → the frame is rasterized into the editor's framebuffer → `platform` presents it, only the 16-row bands whose pixels changed. Headless runs (`init_headless` + `step_frame_headless`) execute the same frame without a window; the tests drive the UI this way with synthetic mouse and key events, and `Blendity --headless-screenshot` runs console commands and writes a picture. A `FramePacer` (`src/core/pacer.h`) starts frames at most `1 / max_fps` apart without oversleeping [67].

3.3 One edit

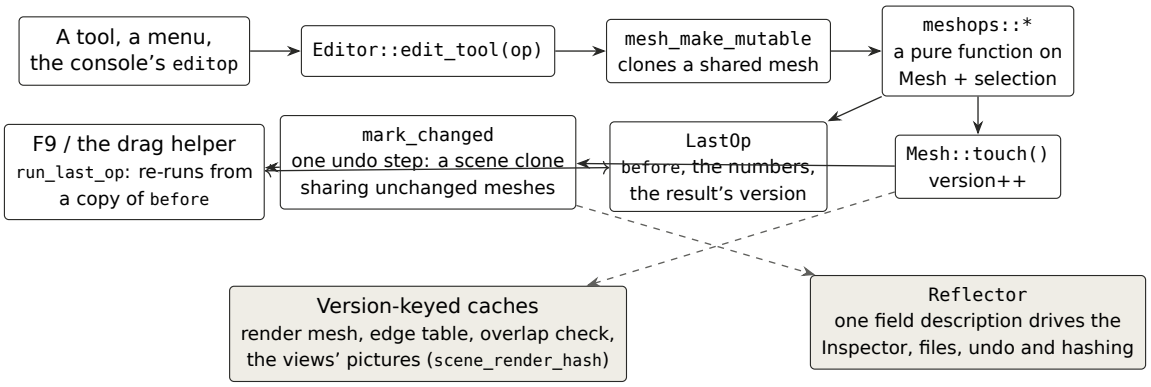


Figure 3.2: One edit, from a tool to an undo step. Solid arrows are the call path; dashed arrows are what the edit's version bump reaches.

A tool or menu calls `Editor::edit_tool(op)` (or the console's `editop`). Operators that take numbers become a `LastOp`: `run_last_op` copies `LastOp::before`, applies the `meshops` function with the current values, records the result's pointer and version, and the F9 panel or the drag helper re-runs it when a value changes. `mark_changed` turns the edit into one undo step: a scene clone sharing the unchanged meshes (ADR 0003).

3.4 Invariants

What every change must keep

- **Copy-on-write meshes** (ADR 0003): `mesh_make_mutable` before writing; re-read `MeshPtr` after an edit; every change `touch()`es the mesh so version-keyed caches refresh. A redoable operator's `LastOp::before` is the undo step's pointer, never written: `run_last_op` makes the operator's one copy (ADR 0053).
- **Edit Mode selections** are vectors indexed like the mesh (`vert_sel_`, `face_sel_`) plus a set of edge keys (`edge_sel_`); any operator that renumbers faces or vertices rebuilds them.
- **Adjustable operations** re-run from `before`; anything that changes the result after it was recorded must update `LastOp::result_version` (ADR 0005), or F9 treats the operation as gone.
- **Geometry robustness** (Q-01): operators leave a structurally valid mesh or refuse with a message; closed meshes stay closed; no new faces lying on top of each other without an explanation (Q-02, counted by the stress suite).
- **Isolation of user files** (C-04): tests and stress run in a scratch project and trash (`BLENDITY_PROJECT`, `BLENDITY_TRASH`); nothing they do touches `Assets/`.
- **No main-thread stalls**: heavy work (bakes, remeshes, skin binds, Live Link's payloads, texture evaluation) runs on workers with a cancel flag and shows in the job tray; the views keep their last picture meanwhile (ADR 0039, ADR 0095).

- **Every tool is reachable from the console** (`editop`, `edittool`, `drawpoint`, …), so tests and scripts can drive it, and `tools/test_coverage_map.py --check` finds no command, keymap action or window without a test (ADR 0056).

3.5 Module map

`docs/ARCHITECTURE.md` keeps the full map, one row per subsystem with the Blender source file it mirrors and the Unity equivalent; this is the shape of it.

Blendity	What it does	Blender source it mirrors
`core/math.h`	vectors, quaternions, column-major matrices; Unity's left-handed Y-up, ZXY Euler	`blenlib/BLI_math_matrix.hh`
`core/jobs.*`	`parallel_for`: oneTBB when available, else its own pool	`blenlib/BLI_task.hh`
`render/canvas.*`	UI drawing, anti-aliased TrueType text from OS fonts	`blenfont/intern/blf_glyph.cc`
`render/raster.*`	tile-binned multithreaded rasterizer, z-buffer, id buffer, outlines	`draw/intern/draw_manager.cc`, `gpu/intern/gpu_select.cc`
`scene/mesh.*`	`positions` + `face_offsets` + `corner_verts` (n-gons), render cache	`makesdna/DNA_mesh_types.h`, `blenkernel/BKE_mesh.hh`
`mesh_make_mutable`	copy-on-write sharing	`blenlib/BLI_implicit_sharing.hh`
`meshops::*`	extrude, inset, bevel, bridge, loop cut, knife, dissolve, fill, spin, slice, push/pull, …	`bmesh/operators`, `bmesh/tools`, `editors/mesh`
`scene/uv*.cpp`	LSCM, ABF++ by parts, Smart UV, projections, packing	`geometry/intern/uv_parametrizer.cc`, `uv_pack.cc`
`scene/material.*`	a Unity-style Material evaluating the Principled BSDF	`nodes/shader/nodes/node_shader_bsdf_pr`
`scene/scene.*`	`GameObject` + `Transform`, components, the `Reflector`	`DNA_object_types.h`, RNA (`makesrna`)
`scene/skin.cpp`, `ik.cpp`	bones, skins (linear / dual quaternion), IK (law of cosines, CCD)	`ikplugin/intern/iksolver_plugin.cc`
`scene/animation.cpp`, `sequence.cpp`	clips of float tracks bound by path, tangent modes; sequences with crossfades	actions / F-curves, the NLA
`render/pathtracer.*`	SAH BVH with instancing, NEE, GGX VNDF, MIS, Russian roulette, denoisers	`intern/cycles/bvh`, `kernel/integrator`
`render/shading.*`	GGX + Lambert, image-based lighting, toon, lilToon, view transforms	`draw/engines/eevee/shaders`, `imbuf/intern/colormanagement.cc`
`render/sky.*`	the Hosek–Wilkie sky driven by the first Directional Light	`intern/cycles/kernel/svm/sky.h`, `intern/sky`
`editor/ui.*`	immediate-mode widgets, popups, menus, tooltips	`editors/interface`
`editor/editor.cpp`	the dock tree: tabs, split / merge areas, layouts, secondary windows	`editors/screen`
`editor/scene_view.cpp`	navigation, gizmos, picking, Edit Mode, the active tool	`editors/space_view3d`, `editors/transform`
`editor/keymap.cpp`	actions, presets, contexts, the editor feel	the keymap editor and presets
`editor/learn.cpp`, `lessons.cpp`	45 lessons linking Unity, Blender and the books	–

3.6 Key conventions

- **Coordinates:** Unity's left-handed, Y-up space [67]. OBJ import and export mirror X and reverse the winding, as Unity's importer does; Live Link and glTF mirror X on the way out. Blendity keeps Unity's axes: a scene opened from Unity is never mirrored.
- **Front faces:** `cross(b - a, c - a)` points outward; faces wind clockwise on screen in a Y-up view.
- **Undo:** a scene-graph snapshot after each finished interaction; meshes shared and copied on first write; clips, materials and textures as text in the undo state.
- **Caches key on content**: `RenderMesh::serial` is new for every build and every in-place deform and never reused; the views' pictures key on `scene_render_hash` with a per-object memo (ADR 0009, ADR 0015); a texture's planes key on the stack's hash.
- **Reflection**: one field description per component (`Reflector`) drives the Inspector, the scene file, the undo hash, the console's `set`, Live Link's material fields and Unity's write-back; a field skipped by hashing (composition guides) never re-renders a view.

Chapter 4

The scene, meshes, undo and assets

4.1 The mesh

`Mesh` (`src/scene/mesh.h`) keeps Blender's layout [87]: `positions`, `face_offsets` and `corner_verts`, so a face is a range of corners and n-gons are first-class; per-corner UVs, per-face material indices, seams, sharp edges, loose edges, custom normals, vertex groups and shape keys are named attributes beside them [69]. The render cache splits corners into render vertices only where their attributes differ (`build_render_vertices`, bit-identical in parallel and serial), builds triangles per face range and MikkTSpace tangents [40]; a positions-only change follows the kept topology in place (`update_in_place`, ADR 0034), with the deforming tail of a modifier stack re-run on a kept mesh (ADR 0022).

Meshes are shared through `MeshPtr` as Blender's implicit sharing does (ADR 0003). An undo step is a clone of the scene that shares unchanged meshes; code that edits a mesh calls `mesh_make_mutable` first; every change bumps `Mesh::version`; caches key on pointer + version or on `RenderMesh::serial`. Holding a `Mesh&` across an operation that may replace the mesh is a bug reviewers look for.

4.2 The operators

`meshops` (`src/scene/mesh.cpp`, `mesh_tools.cpp` to `mesh_tools5.cpp`) is the Edit Mode toolbox, each a pure function on a mesh and selection vectors: extrude (faces, regions by wedge, edges, vertices), inset (individual and region with a continuous limit), bevel with a superellipse profile and clamp overlap, bridge with a Hermite tube, loop cut, knife, connect, dissolve (edges, limited, degenerate), collapse, merge (at centre, by distance, selected-to-unselected), fill and smart fill, poke, triangulate and tris-to-quads, Catmull–Clark [8], Taubin smoothing [57], spin, slice, follow, shell, draft, radial array, fillet, grid, pipe, array, taper, recess, and the SketchUp family: push/pull with its limits, flush steps and holes (ADR 0004), push through, attach face, imprint across faces, repair rings. `src/scene/mesh_internal.h` holds the shared topology helpers (face builder, edge-faces table, the `annulus` ring builder).

Robustness is a requirement (Q-01), not a hope: the stress suite (`stress/stress_main.cpp`) runs the operators on odd meshes (degenerate faces, non-manifold edges, open surfaces, extreme scales, NaN) and counts problems and faces on top of each other (`overlapping_faces`, by Sutherland–Hodgman clipping [56]); the counts may not rise without an explanation in the CHANGELOG.

4.3 Objects and components

As in Unity, everything in a scene is a `GameObject` with a `Transform`; what it does comes from components: `MeshFilter`, `MeshRenderer`, `Light`, `Camera`, `Rigidbody`,

`Animation`, `Armature`, `Bone`, modifiers (`SubdivisionSurface`, `Mirror`, `Array`, `Boolean`, …, ADR 0022), `CameraFilters` (ADR 0008), `ProbeVolume`, `ReflectionProbe`, `SpringBone` (ADR 0049), `Collider`, `LODGroup`, `AvatarDescriptor` and VRChat's others (ADR 0100), and `EngineComponent` for what another engine's file held that Blendity does not interpret (ADR 0101). A component says what the views read of it (`render_role`), so the scene hash takes one virtual call instead of a `dynamic_cast` per type [68].

4.4 The Reflector

A component describes its fields once, in `reflect(Reflector&)`: name, type, range, help, groups, typed object and asset references. That one description draws the Inspector (with essentials first, multi-object editing and expressions in fields), writes and reads the scene file (tolerant of unknown fields, so files move between versions), hashes the component for change detection, answers the console's `set Component.Field value`, keys any field from the Animation window (round 335), and carries fields over Live Link and into Unity's write-back. It is Blender's RNA [87] and Unity's `SerializedProperty` in one. Lists of effects or layers (a camera's filter stack, a texture's layers) are reflected through a prefix per element, so files, undo and hashing work through the same path.

4.5 Scene IO and assets

Scenes are a text format (`src/scene/scene_io.cpp`): one line per object and field, materials and textures by asset path with values kept as a fallback, lightmaps and probes beside the scene (ADR 0010). Material assets (`.mat`) and procedural textures (`.ptex`, ADR 0017) are shared instances written back after each finished change; after loading, the file wins. The Project window moves files on disk and retargets every reference (the asset library, texture paths, scene files, clips); deletes go to the OS trash; moves and deletes are recorded so an undo that brings back an old path follows the file. Imported models (OBJ, FBX through ufbx, glTF, STL, PLY, USD, PMX, VRM, BVH, Collada, `.blend` through Blender) arrive with materials as `.mat` assets, textures found or extracted, skins and clips (ADR 0047); exports share one preparation step (world transform, axes, triangulation, split normals, baked textures) and the writers for each format [58, 74, 80].

4.6 Undo

`UndoState` snapshots the scene graph after each finished interaction (`commit_undo`, with a label the History window lists); meshes are shared and copied on first write; a sculpt or paint stroke is a lighter delta step of the vertices or the 64×64 texture tiles it touched (ADR 0074, ADR 0081); the Draw tab's paper and the reference board keep histories of their own. Vertex groups and custom normals are mended by position at each commit so a topology change does not lose them (ADR 0024). Undo covers the scene, materials, textures and clips as one step per operation (Q-05), and the budget of kept steps is bytes, oldest out (ADR 0026).

Chapter 5

Rendering

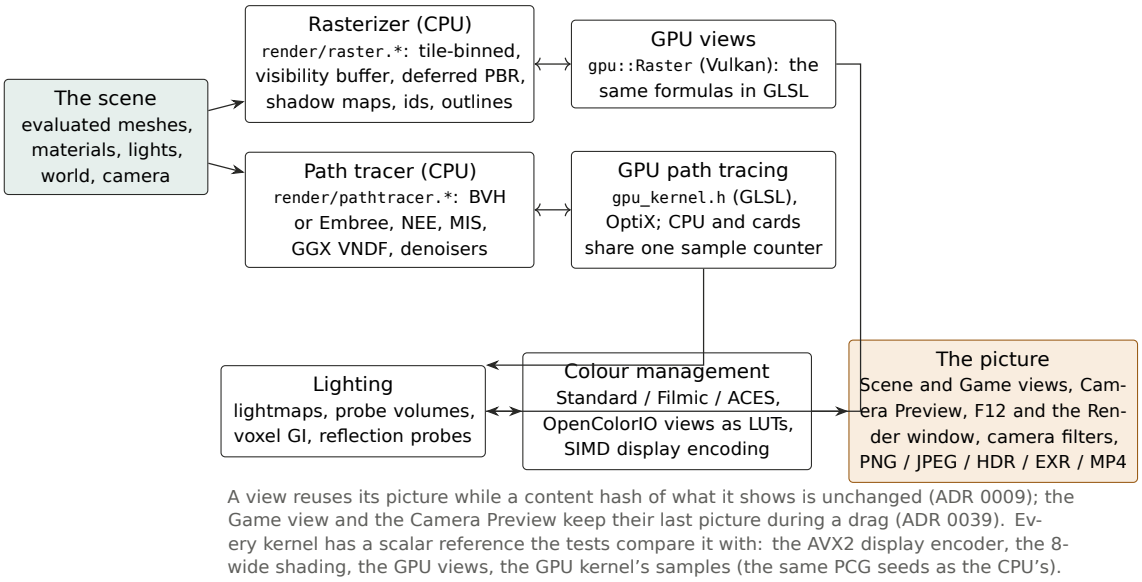


Figure 5.1: The two render paths and where they meet. Every GPU path mirrors a CPU path with the same formulas, and the CPU path is the reference the tests compare it with.

5.1 The rasterizer

`render/raster.*` is a tile-binned, multithreaded software rasterizer with a z-buffer, an id buffer for picking and outlines [69, 68]. Its `Deferred` mode writes a visibility buffer and shades once per pixel: GGX + Lambert [10, 52], image-based lighting (SH9 irradiance [50], mip-filtered specular with the split-sum approximation [32, 23]), sun shadow maps with PCF kept per caster set, area lights as emissive planes with an exact unshadowed integral [21], toon and lilToon responses (ADR 0020, ADR 0100), lightmaps, probes and voxel GI ranked by `light_surface`. Eight same-triangle pixels of a plain material are lit at once with AVX2 (ADR 0035); triangle setup rejects eight at a time; the scalar path is the reference and the tests compare them.

5.2 The path tracer

`render/pathtracer.*` is progressive: a two-level binned-SAH BVH with instancing [61, 41, 15] (or Embree's), next-event estimation of sun, point, spot, area and mesh lights with multiple importance sampling, GGX sampling of visible normals [20, 62], a thin lens with bladed apertures [54], stratified samples [11], Russian roulette, clamping, path guiding through Open PGL [51], volumes (fog inside a closed mesh, ADR 0092), and two denoisers: OpenImageDenoise when it is there, else an edge-avoiding À-Trous

filter [13]. Transparent renders count coverage; filtered renders carry depth and id planes so camera filters run on traced pictures too (ADR 0016).

5.3 The GPU

`render/gpu_kernel.h` is the megakernel in GLSL, a line-for-line port of the CPU's trace with the same PCG seeds, so GPU and CPU samples are interchangeable; `gpu_device.cpp` opens the Vulkan loader at run time, compiles the kernel with shaderc (the SPIR-V cached on disk, pre-warmed by the build), uploads the scene as hashed sections and renders sample batches (ADR 0013, ADR 0014). Devices open in the background and a render adopts its card when ready (ADR 0021); a worker per card keeps two submissions in flight (ADR 0052); CPU threads and every GPU claim sample indices from one atomic counter, the CPU stopping when the cards would finish sooner. The OptiX device (ADR 0037, ADR 0054) runs the same kernel translated to CUDA with rays through `optixTrace`, its PTX committed so the build needs no CUDA toolkit. The views are drawn by `gpu::Raster` (ADR 0036, ADR 0061): a shadow pass, a G-buffer and full-screen lighting with the CPU's formulas, the Edit Mode cage, textures, toon maps, lightmaps and probes read per visible pixel, overlays recorded in the world and replayed on the CPU when the card cannot; a window is presented by the card one frame behind with only the changed tiles taken up. Each window's device is a preference and the fall-back to the CPU is said once (a CPU badge in the Scene view's bar).

5.4 Colour management

HDR → view transform → 8-bit sRGB runs in `render/display.cpp` at 4 or 8 pixels a step with an exact sRGB table (correctly rounded on 17 million floats in the test). The views are Standard, Filmic, Blendity's ACES curve, Unity's ACES (Hill's fit of the RRT + ODT, round 405) [22, 71], and with OpenColorIO Blender's config baked into 97^3 tetrahedral LUTs over a log2 shaper, which the GPU views apply as the same table. Light colour temperature follows the Planckian locus [31].

5.5 Lighting

Global illumination like Unity's (R-11, R-12): baked lightmaps (charts sized from world area, shelf-packed pages, texels baked through the path tracer, saved beside the scene, ADR 0010); probe volumes placed adaptively in bricks of 4^3 ambient cubes [38] with validity, virtual offset and dilation (ADR 0012); voxel GI [59, 3, 12, 19] for one bounce of the sun and sky occlusion in the rasterized views with nothing to bake (ADR 0011), on the GPU as a compute kernel; realtime lightmaps and live probes that gather through the voxel grid; reflection probes (round 401); mesh maps (normal, position, curvature, occlusion, thickness) baked into the UV layout (ADR 0079); texture bakes that turn a material into plain images (ADR 0019); and the world's Hosek–Wilkie sky [24], HDRIs, sun disc, horizon band and fog. A scene opened from Unity shows Unity's own lightmaps, probes, reflection probes, fog, skybox, post-processing and BRDF weight as Unity shows them (ADR 0098, ADR 0099; rounds 400–405), measured against Unity's renders at fixed stations (`tools/unity_compare.py`).

5.6 Camera filters

Filters are components on a camera's object (ADR 0007, ADR 0008): an ordered stack of effects, each contributing raster settings (vertex snap, affine UVs, texture rules, screen-door transparency: the PS1, N64, Saturn and DOS looks [79]), an internal resolution, and image passes in linear or display light (colour grading in ASC CDL order [72], bloom [29], vignette, chromatic aberration, lens distortion [6], film grain, CRT, pixelate, edge outline, sharpen, halftone, hatching, line art, paper, colour ramp, and the Anime and Manga looks). The same stack runs on the Game view, the Camera Preview, F12 and camera sequences; the Scene view can look through it. Filters look the same at every picture size: pixel-sized parameters scale with the height.

Chapter 6

The editor

6.1 The toolkit

`editor/ui.*` is an immediate-mode toolkit (`ui::Context`): buttons, fields with expressions (`+=1`, `*=2`, `L(0,10)`, `R(0,1)`), dropdowns, colour fields and the Color window, popups, menus, tooltips, tab strips, toasts and the job tray, drawn by the canvas (`render/canvas.*`: anti-aliased TrueType text from the OS fonts; icons rasterized once per size). Every widget records its rectangle, which is what the Fitts audit reads and what the coverage map's `--widgets` counts. Text contrast is measured per theme against WCAG's 4.5:1 [60]; polygroup and signal colours follow Okabe and Ito's colour-universal set [45].

6.2 Docking, windows and the module registry

`editor/editor.cpp` holds a dock tree of areas and tabs: drag a tab onto a tab bar to group it, onto an edge to split, out of the editor for a window of its own on any display (ADR 0059); a layout is a dock string plus a focus (the tool groups a stage shows, ADR 0027), and the user's layouts and the whole arrangement go into a `.blendity-ui` file. The module registry (`editor/modules.cpp`, ADR 0073) is one row per window kind: title, icon, group, the stages it belongs to, whether a second window may hold one, whether its group folds, its shortcut. The 26 windows are Scene, Game, Hierarchy, Inspector, Project, Console, Learn, Research (retired), Profiler, Render, UV Editor, Materials, Modeling Tools, Lighting, Textures, Rigging, Animation, Sequencer, Live Link, Editing, My Tools, Draw, Layers, History, Board and Avatar; the tab strips, the Window menu, the layouts and the tests read the registry, so a new window is one row.

6.3 Keymaps, feels and the modal pattern

`editor/keymap.cpp`: `shortcut_actions()` lists every action (id, label, category, context); `keymap_preset(name)` builds the bindings for Unity, Unreal, Blender, Maya, 3ds Max, SketchUp and Adobe; `set_editor_feel` sets the preset and the Scene view's mouse navigation together; the user's overrides sit on top; `handle_shortcuts` matches a key against the most specific active context first (Scene + Edit, Edit, Scene, anywhere). Every shortcut label in a menu or tooltip comes from the keymap through `shortcut_text`, so no label can lie under another preset (ADR 0085). Every edit operator follows one modal pattern (`docs/USABILITY.md`): a key starts it, the mouse or a typed number shapes it, X / Y / Z lock an axis, Ctrl snaps, a click or Enter confirms, Esc cancels leaving no undo step, F9 adjusts afterwards as the same step. The Wheel (ADR 0068) is one radial menu for Space, the pen's button or a finger held still, with a set per mode the user fills from any menu; My Tools panels and Assign Shortcut give any command a button or a key (ADR 0069).

6.4 Tools

The Scene view runs the active tool each frame: the Draw tools with SketchUp inference and guides (`draw_tool.cpp`; the Materials window's preview spheres are Blinn–Phong [48]), the Knife, Push/Pull as a modal operator, G / R / S (`modal_transform.cpp`), the gizmo with Shift-extrude and origin editing, the sketch pen with its guides, canvases and the camera solved from the perspective guide (ADR 0063–ADR 0067, [26, 9, 18]), Sculpt Mode (ADR 0074–ADR 0078, [27, 17, 25]), Paint Mode by projection (ADR 0081, [4]), Weight Paint, Strokes and Scatter (ADR 0084), Puppet Posing's handles with a learned pose prior and exact IK (ADR 0087, [46]), and the VR View's controllers through OpenXR (ADR 0096, [127]). Fingers and pens come through one pointer model (ADR 0062): in Tablet Mode fingers navigate and the pen works; on Linux through XInput 2; on a Mac the trackpad.

6.5 The console and headless runs

Every tool is reachable from the Console's command line (`editor_commands.cpp`; `help` lists about 240 commands), and that is the editor's scripting surface: the tests, `--headless-screenshot --cmd`, the HCI production runs, `tools/unity_compare.py` and `unity_e2e.py` all drive it. The console recalls its history, answers an unknown command with the nearest known ones, and since round 310 no verb an artist uses lives only there: a test keeps every one reachable from a menu, a button or a field.

6.6 The Learn tab

`editor/learn.cpp` renders 45 lessons from `lessons.cpp`, each connecting a Unity concept, the Blender concept with the source folder it lives in, and the theory in the reference books [69, 67, 68], with Try-it buttons that run console commands, questions that reveal their answers, and a References lesson of the papers behind the features. Every new feature adds a lesson; the user guide names the lesson for each chapter, and Index C of that guide lists each lesson's sources.

6.7 Where the HCI work lives

ADR 0085 set the methods (Nielsen's heuristics [44, 70], Fitts's law [16], the keystroke-level model [7, 65], cognitive walkthroughs [49], transfer of learning, WCAG) and the foundations every area uses: toasts with a Fix and disabled buttons that say why, one job tray, status hints from a table, labels from the keymap, unsaved-changes prompts, a History window, a colour-vision-safe palette and a High Contrast theme. The measuring sticks are in the test suite (the Fitts dump, the coverage map, the label audit, the hint completeness test, the contrast test) and the artefacts in `docs/hci/` (walkthroughs, the KLM table recounted by `tools/hci_klm.py`, the comparison reports against Blender, Unity, Godot, Unreal, Adobe and VRChat's limits, the 118-task checklist, the study protocol written but not yet run [5]).

Chapter 7

Bridges, Live Link and file formats

7.1 Live Link

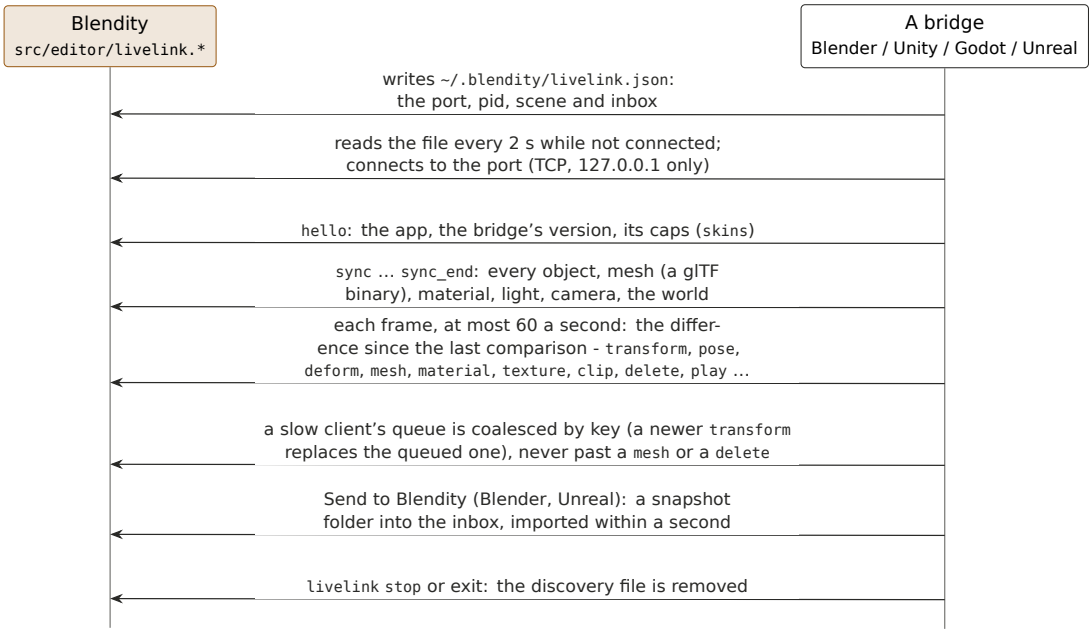


Figure 7.1: The Live Link sequence (`docs/LIVELINK.md`, ADR 0055, ADR 0086). One way out: nothing a client sends is read but its hello.

Live Link sends the scene open in Blendity to other programs on the same computer as it changes. The transport is a TCP server on 127.0.0.1 (`src/core/net_server.*`, no library); the framing is one JSON object a line with a binary attachment of N bytes when the line ends with `"bin":N`; meshes go as glTF 2.0 binaries [58] from Blendity's own exporter in glTF's axes (right-handed, Y up, metres: X mirrored), materials as glTF material objects with the toon fields as an extension, textures by their pixels, skins once at rest and then poses and weights, clips baked as the engines' own, colliders and LOD levels, the world and the camera's look, VRChat's components where the SDK is there. What changed is found by comparing the scene with what was last sent, once a frame while a client is connected, so a drag streams while it happens and undo, Stop and a reload need no code of their own; each client has its own queue and sender thread, coalesced by key so a slow client skips to the present. With four engines connected, a 100,000-vertex mesh changing and a 50-bone rig posing every frame, it takes a millisecond or two of a frame (round 382).

7.2 The bridges

`bridges/` holds one package per engine, Blendity's own code under the same GPL: a Blender add-on (Python), a Unity package (`com.blendity.livelink`, C#, with VRChat stand-ins and SDK checks), a Godot add-on (GDScript) and an Unreal plugin (C++), each with a smoke test, an auto-connect test and a live session test, packaged by `tools/package_bridges.py` and installed from Window > Live Link > Install a Bridge. The Blender and Unreal bridges also send to Blendity: a snapshot folder into the inbox the discovery file names, imported within a second. `tools/livelink_fake_server.py` serves a recorded session for a bridge's tests, and `livelink_dump.py` is the reading loop in fifty lines of Python.

7.3 File formats

Export shares one preparation step and a writer per format (`src/scene/export.cpp`): OBJ + MTL, FBX 7.4 as a record tree with Blender's binary encoding [74] and an ASCII one, glTF 2.0 (`.glb` or `.gltf`, embedded or copied textures, morph targets, skins and sampled channels), STL, PLY, USD text [80]; options as Blender's (Selected Only, Apply Modifiers, Scale, Up Axis, Triangulate, Keep Transforms, Armatures and Animation, Colliders and LOD levels named for Unity, Unreal or Godot). Every format was checked both ways against Blender 5.2 (R-06). Import reads the same formats plus MMD's PMX / PMD / VMD [134], VRM [130], BVH, Collada, and `.blend` through the installed Blender, with an Import Preview that shows the model under a chosen shading before anything is written (ADR 0047).

7.4 Unity's project files

Rounds 369–408 made Blendity read and write Unity's own files: `.unitypackage` unpacked into the project with GUIDs kept; prefabs and scenes brought in with nested prefabs and overrides, every other component as an Engine Component editable under Unity's names (ADR 0101); `.mat` files converted by shader (Standard, URP, lilToon, Poiyomi, MToon, UTS2, Filamented); a `.unity` opened as the scene with the look Unity gives it (fog modes, skybox, camera flags, post-processing, lights' shadows, reflection probes, ADR 0099; Unity's own lightmaps and probes read by the installed Unity in the background, ADR 0098); VRChat's components as Blendity's (ADR 0100); Unity's `.anim` clips read and written, the Unity file never written over (ADR 0102); humanoid retargeting as Mecanim reads a T and an A pose (ADR 0103); Export Unity Package writing the work back under the same GUIDs. The parity is measured: `tools/unity_compare.py` renders the same stations in Unity (batch mode) and in Blendity and scores them (MAE, PSNR, SSIM, a luminance grid's correlation), and `tools/unity_e2e.py` runs the whole fresh-project flow; an avatar passes at every station, a world is near the target (`docs/hci/compare-unity-vrchat-2026-11.md`).

7.5 Godot, Blender, Unreal

Godot scenes (`.tscn`) come in with their nodes, primitives, materials, lights, cameras and collision shapes and go back with the objects' transforms in their own lines, scripts kept (round 373, [143]). Blender files come in through the installed Blender's glTF

exporter with lights, cameras, actions and custom properties (round 374). Unreal's bridge sends a level through its FBX exporter with each actor's place and Blueprint class ([142]).

7.6 MoonRay

DreamWorks' MoonRay [91] is an optional external renderer (ADR 0041): built by the `moonray.yml` workflow on Linux, run as a program of its own (in WSL on Windows), fed a scene written by Blendity, its picture read back into the Render window. It is never linked and never shipped with Blendity.

Chapter 8

Research features

8.1 The research folder

`research/` connects papers to the editor: `papers/` is where the user drops them, `FEATURES.md` is the tracker (paper → feature → status → source file → test), and `src/research/` is where a paper's method is implemented in plain C++ with no new dependency and registered with `research::register_feature`, which puts it in Learn > References, Mesh > From Papers and the Inspector's MeshFilter tools with its citation. The rule is that the paper is cited in the code's comments, in the Learn tab and in the tracker, and that the status says what was taken from the paper and what was not.

8.2 What the papers became

The tracker's rows, with each paper's key in the References chapter (Index A lists them again by source file).

Paper	Feature	Status
Taubin 1995 [57]	Taubin $\lambda\|\mu$ smoothing, the example feature	implemented
Catmull and Clark 1978 [8]	subdivision (the Subdivide tool, the Subdivision Surface modifier)	core
Thiedemann et al. 2011 [59]	realtime GI: binary voxel columns, the mip-map ray test, one bounce of the sun through a reflective shadow map, sky occlusion (ADR 0011)	implemented (CPU and GPU)
Chen et al. 2013 [9]	solids from a few strokes: a box from three lines, a cylinder from two circles, a sweep (ADR 0065)	implemented
Gryaditskaya et al. 2020, Hähnlein et al. 2022 [18]	a scaffold of construction lines lifted from the picture plane (ADR 0067); symmetry and curved lines not taken	experimental, gate met
Igarashi et al. 1997 [26]	pen strokes corrected at the lift: split at corners, fitted, pulled to the guides (ADR 0063)	implemented
Jacobson, Kavan and Sorkine-Hornung 2013 [27]	DynaMesh's inside test by generalized winding numbers along grid lines (ADR 0076)	implemented (the grid-line form)
Gibson 1998 [17]	DynaMesh's surface: naive surface nets over the sign grid (where marching cubes [35] would tessellate), then relaxation	implemented
Huang et al. 2018 [25]	Quad Remesh through Blender's patched QuadriFlow on a worker (ADR 0077)	integrated (library)
Baxter, Wendt and Lin 2004 [4]	the painterly brush's wet mixing: a one-value pickup, not the paper's paint volume (ADR 0083)	simplified
Abdrashitov et al. 2023 [1]	automatic weights and weight transfer by inpainting (ADR 0028)	implemented
Oreshkin et al. 2022 [46]	Puppet Posing's pose prior: a residual network trained with goals hidden; much smaller, positions only (ADR 0087)	implemented
Okabe and Ito 2008 [45]	sculpt polygroups in the eight colour-universal colours	implemented
Khronos 2021 [58]	skinned meshes exported with joints, inverse bind matrices and sampled channels	implemented
W3C 2018 [60]	the contrast ratio measuring every theme; the High Contrast theme at 4.5:1	implemented
Nielsen 1994, Fitts 1954, Card et al. 1980, Polson et al. 1992, Brooke 1996 [44, 16, 7, 49, 5]	the HCI audits, the KLM table, the cognitive walkthroughs, the study protocol (ADR 0085)	applied (methods, not algorithms; the study not run)

Beyond the tracker, the renderers and the solvers cite their sources in the code and in the Learn tab's References lesson: the microfacet model and its sampling [10, 62, 20, 52], area lights [32, 21], the disk mapping and stratification [54, 11], the BVH and the ray–triangle test [61, 41, 15], path guiding [51], the À-Trous denoiser [13], the sky and irradiance maps [24, 50], colour temperature [31], noise [47], LSCM and ABF++ [34, 53] with PartUV's part-based search [63], the cotangent Laplacian [39], MikkTSpace [40], the teapot [43], CCD inverse kinematics [64], the spring and strand solvers [28, 33, 36, 37, 14, 2], toon and anime rendering [42, 30], and quaternion interpolation [55].

Chapter 9

The software development lifecycle

Blendity is written with an AI under a person's direction and review. This chapter is the lifecycle as it is practised, with every artifact named by its path, so that someone can repeat it or audit it.

9.1 The roles

- **The user** (Emiliano Manalo) decides what Blendity is for, which features it gets and in what order, approves each specification and decision record before code is written, reviews every pull request on GitHub and merges it, and decides when a release is cut. Nothing lands without that review (C-05, ADR 0006).
- **The main session** (Claude through Claude Code [85]) plans: it reads `docs/requirements.md`, `docs/ARCHITECTURE.md` and the decision records, writes the task spec and any ADR, implements the hard, cross-cutting or geometric work itself, runs the suites and the audits, writes the documentation and opens the pull request.
- **The `implementer` agent** implements a routine, well-specified task from its spec, builds and runs the unit tests; it makes no architectural decisions, adds no dependency, never commits.
- **The `test-engineer` agent** writes independent tests from the spec's acceptance criteria, edge cases first; it edits only the test folders and reports bugs as failing tests.
- **The `code-reviewer` agent** reviews the diff against the spec, the ADRs and `CLAUDE.md` and reports ranked findings with concrete failure scenarios and a verdict; it never edits files. Its findings are fixed or answered in the task's record. Agents run one after another, never fanned out.

9.2 The lifecycle

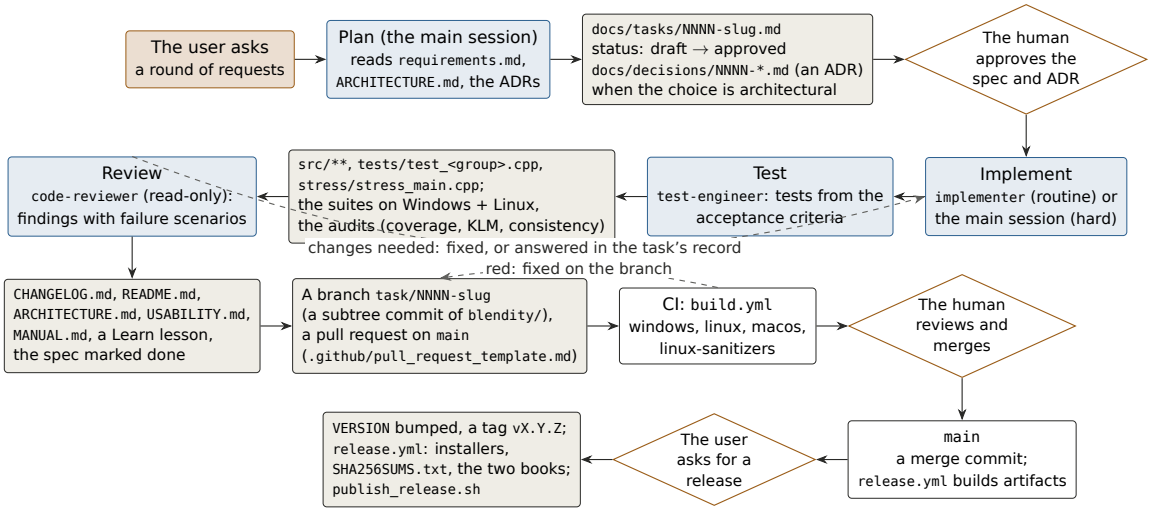


Figure 9.1: One task from the request to the release. Diamonds are the human's gates; the boxes with paths are the artifacts.

1. **Plan.** A round of requests becomes task specs `docs/tasks/NNNN-slug.md` from `TEMPLATE.md`: goal, scope (in / out), likely files, acceptance criteria that a test can check, the tests and stress sections to run, risks and edge cases, the documentation to update. An architectural choice (a subsystem, a data-model change, a dependency, a change to a convention, to how the project is built, tested or released) gets an ADR from the template in `docs/decisions/README.md`: status, date, requirements, context, decision stated so review can check it, consequences. Records are never rewritten; a later one supersedes by number. **Gate: the human approves the spec and the ADR.**

2. **Implement and test.** The implementer or the main session writes the code; every behaviour change gets a unit test in the `tests/test_<group>.cpp` of its subsystem, named for what the user sees, in the round's group at the end of the file; a measurement is a stress row, never an asserted time (ADR 0056). The suites run on Windows and Linux, both builds; the audits run (`test_coverage_map.py --check`, `hci_consistency_audit.py --check`, `hci_klm.py --check`, `unity_meta.py --check`, `make_books.py --check`); geometry changes run the stress sections and their counts may not rise unexplained.

3. **Review.** The reviewer's findings are fixed in a second pass, each with a test, or answered in the task's "Review" section. Some rounds took three passes; the record says so.

4. **Document.** Every change: a CHANGELOG entry (asked / added / changed / checked, with the numbers), the README's count and feature paragraph, an ARCHITECTURE row for a new subsystem, a USABILITY row for a user-facing problem solved, the manual, a Learn lesson with Try-it buttons and questions for every new feature, the spec marked done with its PR link.

5. **Publish.** A subtree commit of `blendity/` is pushed to a branch `task/NNNN-slug` and a pull request opened on `main` with the template (`.github/pull_request_template.md`): the task, what changed, the tests with their counts, the review's verdict, the

documentation boxes. Commits end with the model's `Co-Authored-By` trailer; pull requests say they were generated with Claude Code. A round's PR is stacked on the previous round's branch when the rounds depend on each other, and the body says which PRs a merge also merges.

6. **CI.** `build.yml` builds Windows, Linux and macOS, runs the unit tests and a quick stress pass, takes a headless screenshot, and runs the tests under AddressSanitizer and UndefinedBehaviorSanitizer on Linux; the four jobs are required checks. `main` is protected: pull requests only, no force pushes.

7. **Merge. Gate: the human reviews and merges** with a merge commit, never a squash or a rebase of a stack; when the user asks an agent to merge, only after CI is green on all jobs. `release.yml` builds every merge as artifacts to try.

8. **Release. Gate: the user asks for one.** `VERSION` is bumped, a `v*` tag pushed, and `release.yml` builds the installers, checks them, attaches the books and publishes (chapter 10).

9.3 Traceability

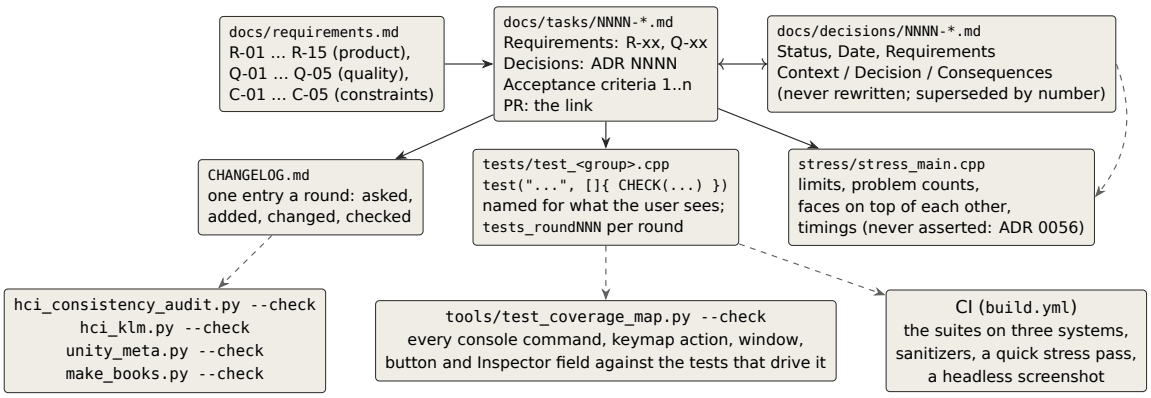


Figure 9.2: From a requirement ID to the tests and the audits that hold it.

A requirement's ID (R-, Q-, C-) is cited by the task spec and the ADR; the spec's acceptance criteria become the tests; the CHANGELOG entry names the task, the tests and the numbers; the coverage map ties every command, action, window, button and field to the tests that drive it; and the HCI tools recount their tables so the documents cannot drift from the counts. Changing a requirement is a human decision recorded in an ADR.

9.4 Engineering standards

`CLAUDE.md` is the contract every session reads first: the project map; how to build and run; the testing rules (real events for UI behaviour, `meshops` calls for geometry, a scratch project, a test that disagrees with the code is evidence); the code standards (C++17 on three compilers, match the surrounding code, copy-on-write, operators through `LastOp`, every tool reachable from the console, no new dependency outside what Blender bundles, an SPDX header in every file, robust geometry); the documentation

duties; the workflow and approval gates; git and publishing rules (commit only when asked, never the user's `Assets/`, never push to `main`, never rewrite history).

9.5 The HCI method in the lifecycle

Usability is designed and measured, not assumed (ADR 0085): each HCI addition names the heuristic or model that motivates it; the baseline is taken before any fix so each round's effect is a diff in git; the keystroke-level estimate for frequent actions went from 80.9 s to 54.65 s over the pass (`docs/hci/klm.md`), the failing walkthrough steps from 24 to 7, wrong shortcut labels to 0; the comparison reports put a VRChat avatar, a VRChat world and a short film made in Blendity beside Blender, Unity, Godot, Unreal and Adobe from their documentation, with the findings ranked for the user to choose from; the 118-task checklist carries target times. The think-aloud study with SUS is written (`docs/hci/study-protocol.md`) and not yet run; the record says so.

9.6 AI disclosure

`docs/AI_DISCLOSURE.md` states, for anyone reading, using or building on the project: the direction is human; the code is written with Claude from approved specs under the standards above; every commit says so; numbers are measured, not guessed, and where something was not tried the record says so; there is no telemetry, no cloud, and no AI at run time except the optional local inbetweening download; what Blendity makes is the user's. Chapter 13 sets out why that is compatible with the GPL.

Chapter 10

Build, test and release

10.1 Building

You need a compiler. `build.bat release all` on Windows finds Visual Studio or the Build Tools; `./build.sh release all` on Linux (g++, libx11-dev, libxi-dev) and macOS (the command line tools); CMake is an optional route. Each build produces `Blendity`, `blendity_tests` and `blendity_stress` in `dist/<platform>/`. With `blender/lib/<platform>` beside the project (fetched by `packaging/fetch_blender_libs.sh` at the commits pinned in `packaging/blender-libs.txt`, about 600 MB) the build links Blender's libraries and copies their runtime files beside the executables; `BLENDITY_NO_LIBS=1` forces the dependency-free build. The two builds are the two test suites the user asked for: both must keep building and passing (C-02). Sanitizer builds need the dependency-free build, as the prebuilt Jolt has no RTTI. The GPU path tracer's shaders are pre-warmed by the build; the OptiX kernel's PTX is committed so no CUDA toolkit is needed.

10.2 The test suites

`blendity_tests` runs every test (`test("what the user sees", []{ CHECK(...) })`) in 16 groups (`--list-groups`, `--group <name>`; `BLENDITY_TEST_FILTER` for one by name): modeling, render, lighting, gpu, assets, animation, physics, editor, livelink, input, sketch, board, sculpt, hci, vr and main. UI tests drive the editor headless with synthetic mouse and key events; geometry tests call `meshops`; every test runs in a scratch project. `tools/run_suites.py` runs every group of every build given as its own process, each with its share of the cores, longest first, and exits 1 on a failure or a crash: both Windows builds in about a minute and a half.

Suite at 0.13.0	Checks	Tests
Windows, with Blender's libraries	15,902	1,543
Windows, dependency-free	15,253	1,543
Linux (WSL), dependency-free	15,259	1,543

The dependency-free build skips the library-only tests (Embree, OIDN, Manifold, Jolt, FFmpeg, EXR) and says so; the counts differ by those. `blendity_stress` is the other suite: limits, naive-against-optimised comparisons, problem and overlap counts, Live Link's cost, the Blender parity rows; `--quick` runs in CI in about a minute, and `stress/results/latest.md` is the committed baseline.

10.3 The audits

`tools/test_coverage_map.py --check` (no command, action, window, button or field without a test), `tools/hci_consistency_audit.py --check` (the interface conventions:

"…" means asks first, Title Case, tooltips as sentences, no command names in visible text, labels that fit), `tools/hci_klm.py --check` (the KLM table recounted), `tools/hci_label_audit.py` (no hard-coded shortcut label), `tools/unity_meta.py --check` (the Unity bridge's `.meta` files complete), `tools/make_books.py --check` (the reference list and index tables not drifted, every entry cited). The HCI sweep (`tests/test_hci.cpp`) walks 826 screens for cut labels, narrow panes and contrast.

10.4 Continuous integration

`build.yml` on every push and pull request: windows, linux (ubuntu-latest) and macos build everything, run the unit tests with each failing check as an annotation, run a quick stress pass under a 15-minute alarm, take a headless screenshot and upload the executables; linux-sanitizers builds the tests with ASan + UBSan and runs them. All four are required to merge. `release.yml` on a `v*` tag, on a merge into `main` and on a pull request touching packaging: builds Blendity-full and Blendity on each system with the libraries fetched and cached by their pinned commits, checks the builds are what they should be (`--probe`), installs and uninstalls on Windows (full and lite types), starts the AppImage and the mounted `.dmg` headless, packages the five bridges, builds the two books (`make_books.py --check --pdf`), and on a tag publishes. `moonray.yml` builds MoonRay for the bridge.

10.5 Packaging and deployment

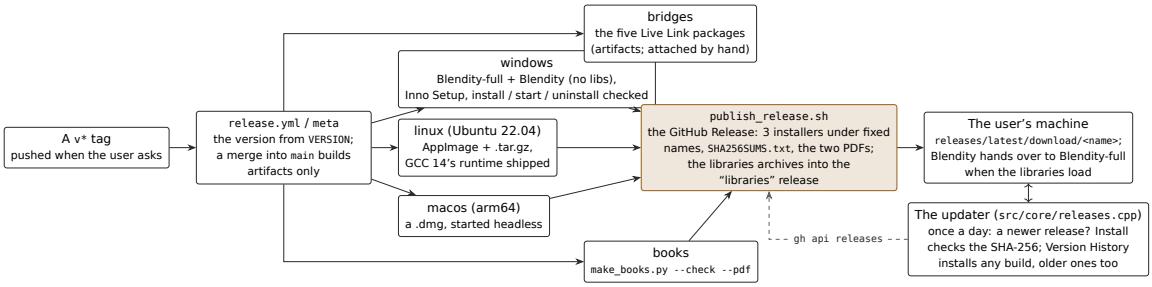


Figure 10.1: From a tag to the installed program and its updater.

Each system gets one installer holding Blendity (dependency-free, always starts) and Blendity-full (with the libraries): an Inno Setup installer for Windows [125] installing for the user alone with a `.scene` association, an AppImage [86] and a plain `.tar.gz` for Linux built on the oldest supported Ubuntu, a `.dmg` for Apple-silicon Macs. `publish_release.sh` publishes the three installers under names without the version (so `releases/latest/download/<name>` always fetches the newest), `SHA256SUMS.txt` for everything, the two PDFs, and the libraries archives into one "libraries" release that Blendity's updater reads; a `0.x` version is titled a pre-release of the program, a suffixed tag (`-beta.1`) a GitHub pre-release. The release notes are GitHub's summary of the merged pull requests followed by `packaging/install-notes.md`.

The updater (ADR 0033) asks GitHub once a day, when online and when the user allows, whether a newer release exists, shows what it brings with Install / Not now / Skip this one, checks the download against the SHA-256 list, saves the scene and installs; Help > Version History installs any build, older ones too. An install without

the libraries offers to download them for this build; Preferences > Libraries is always the user's choice.

10.6 Releasing

A release is the user's decision. The steps: `VERSION` bumped (with the fallback in `releases.cpp`, the installer script's example, the README's pre-release note and the manual's mention), a CHANGELOG entry, a merge, `git tag vX.Y.Z` on the merge and `git push origin vX.Y.Z`; `release.yml` does the rest and the user checks the assets on the Releases page. Version 0.13.0 is the release this book was written for; versions before 1.0 are pre-releases.

Chapter 11

Performance and the minimum machine

11.1 The budget

`docs/PERFORMANCE.md` is the record of fifteen passes, each with its limits found, efficiencies applied, design choices the numbers confirm, and honest negatives. The rule behind them: the editor stays interactive (Q-04), a frame within the budget for ordinary scenes, heavy work off the per-frame path (ADR 0095); a unit test may print a time but never asserts one (ADR 0056); every measurement names the machine.

11.2 The minimum machine

A thin laptop such as the HP Envy 13.3-inch 2-in-1: an Intel i7 with 4 cores / 8 threads (i7-1165G7 class), 8 GB of RAM shared with Iris Xe graphics, a 1920×1080 touch screen at 150 % (ADR 0026). It starts on the Low quality tier and says so: the views drawn at 67 % and scaled up, a 1024 shadow map, 60 fps; editing frames at about 35–40 fps, orbiting at 60. Medium (85 %, 2048, 120 fps) and High (100 %, the scene's shadow map, 144 fps) are one click away; renders always use the scene's own settings. The GPU is for path tracing, baking and realtime GI; an integrated one gets a quarter of the RAM for it. Thread, texture and undo budgets are ceilings the preferences set.

11.3 What the passes found

- **Editor frames** (passes 6, 7): the frame fills only its border strips; icons rasterize once per size; folder listings and the material library rebuild only on change; a view reuses its picture while its content hash is unchanged (ADR 0009); the per-object memo of the scene hash (ADR 0015).
- **Viewport and Edit Mode** (passes 10–12): an in-place update of a render mesh around the moved faces only (ADR 0034), the overlap check on every thread with answers kept by content and a local re-check after moves (ADR 0040), the operators making one copy of a mesh (ADR 0053), a selection changing the selection and not the mesh, undo giving back Edit Mode's selection; a 50k-face mesh enters Edit Mode in about a frame; extrude, undo and entering Edit Mode measured against Blender's own frames.
- **The GPU** (passes 4, 11, 13, 14): the views in one submission (ADR 0051), the cards working between frames (ADR 0052), incremental scene uploads, the window presented by the card with changed tiles only (ADR 0061); OptiX against GPU Compute measured (ADR 0037).
- **Rendering** (passes 2, 3): Blender's libraries against Blendity's own code, each measured (Embree's BVH, OIDN, TBB, Eigen's direct solve for LSCM, Open-

Subdiv); hand-written SIMD for the display encoder and the rasterizer's setup, chosen at run time by CPUID so the executables still run on older CPUs [67].

- **Blender 5.2 as the benchmark** (passes 13, 15): every editor feature timed in Blendity and in Blender headless on the same scenes under the same row names (`tools/bench_blender.py`, `bench_table.py`), Blender's viewport, playback and start-up timed beside Blendity's window.
- **Live Link** (round 382): a millisecond or two of a frame with four engines connected.

11.4 Profiling

`FrameProfile` records where each frame's time goes (the Scene view and its overlays, the Game view, the preview, the shadow map, the UI, the present; the parts inside an Edit Mode drag), shown by the Profiler window, the stress suite and the tests; `MeshCounts` tallies the mesh caches' work process-wide. The Profiler's Budget ranks a scene or avatar against VRChat's PC and Quest performance ranks [151].

Chapter 12

What is not recreated, and known limits

12.1 Not recreated from Blender

Materials are a fixed Principled BSDF with a layer stack of textures rather than a node graph (the Textures window maps Blender's texture and converter nodes to layers; Mix Shader, Light Path and the attribute nodes are not there); there is no node editor, no geometry nodes, no compositor beyond the Editing tab and the camera filters, no Grease Pencil beyond the sketch pen and the Draw tab, no Python API (the console is the scripting surface), no `.blend` reading except through the installed Blender, no Alembic, no caustics or subsurface scattering in the path tracer (glass refracts; light through it reaches surfaces only along paths, as Cycles without caustics), no hair system beyond strands as geometry and spring chains.

12.2 Not recreated from Unity

No scripting runtime (Play has Rigidbody, Rotator, Oscillator, PlayerController, Animation and the Sequencer's triggers, not C# scripts); no Animator state machine (sequences and triggers instead, ADR 0030); UI canvases and particle systems from a Unity scene are kept as data and not drawn; Unity's Avatar asset is not read (the humanoid map finds the parts by names and shape); Mecanim's muscle space is not implemented (a rig whose bones roll differently carries the roll with the turn); fingers, the face and IK are not retargeted.

12.3 Known limits worth knowing

- The dependency-free build has no Boolean, Decimate, EXR, OpenSubdiv, Open PGL or MP4 of its own; it offers Blender's libraries in a click.
- GPU path tracing and the GPU views do not draw volumes or the Principled extras (Subsurface, Sheen, Coat, Anisotropic); such scenes go to the CPU and say so.
- An OpenColorIO view saved by index can land on a different view when the view list changes; the fixed views come first for that reason (round 405).
- The think-aloud study (`docs/hci/study-protocol.md`) has not been run; the HCI numbers are expert-path estimates and audits, not sessions with participants.
- Macs with Intel processors are not supported; macOS is built and tested only on CI.
- Sound in the Sequencer does not play on macOS (the render still carries it).

- The humanoid map reads world directions and cannot read an arm with a helper bone inside its chain.

Chapter 13

Licensing and credit

Blendity follows Blender's design closely and is licensed to be compatible with it, with the libraries Blender bundles, and with the way it is written: by an AI under a person's direction. This chapter states the licence terms once, the reasoning behind them, and who is credited for what.

13.1 The terms

The licence, in four lines (ADR 0002, ADR 0104)

- **The source** is GPL-2.0-or-later [75], the same as Blender's [87]. Every source file carries `SPDX-License-Identifier: GPL-2.0-or-later` [82]; the full text is the root `LICENSE`.
- **The executables** are distributed under GPL-3.0-or-later [76], because they combine that source with Apache-2.0 code [81] (MikkTSpace, the Cycles sky evaluation, and when built with Blender's libraries Embree, OpenImageDenoise, oneTBB, Manifold, Open PGL and others), and Apache-2.0 combines with GPL version 3 but not with version 2 alone [77]. This is Blender's own situation.
- **Your work is yours.** The GPL covers the program, not its output: scenes, meshes, textures, renders, animations and exported files belong to whoever made them.
- **Third-party notices** ship with every binary (`licenses/`), and any new copied code or library adds its row to `licenses/README.md` in the same change.

13.2 Why these terms are compatible with Blender's

Blender is GPL-2.0-or-later. A program that borrows its design and ports pieces of its source must be under a licence the GPL accepts, and "GPL-2.0-or-later" for the source keeps the two interchangeable in both directions: code could move from Blender's tree into Blendity's and back without a relicensing. The "or later" clause is what lets the binaries be GPL-3.0-or-later: Blender's own release binaries bundle the same Apache-2.0 libraries and are distributed on the same basis.

Three kinds of third-party material are in or near the tree, each handled differently:

- **Code copied unmodified from Blender's tree** into `extern/`: ufbx and fast_float (MIT), MikkTSpace (Apache-2.0), the Hosek–Wilkie sky (BSD-3-Clause), QuadriFlow with LEMON and pcg32 (BSD-3-Clause, Boost, Apache-2.0), cuew (a modified Apache-2.0), `libc_compat`. Their licence texts are in `licenses/` and `extern/README.md` says where each came from.

- **Code ported, not copied**: `src/render/sky.cpp` re-implements the sky radiance evaluation of Cycles' `intern/cycles/kernel/svm/sky.h` (Apache-2.0, 2011–2022 Blender Foundation) in Blendity's own functions. The file's header keeps that notice word for word, its SPDX line names both licences (`GPL-2.0-or-later AND Apache-2.0`), and the file as a whole is distributed under GPL-3.0-or-later through the "or later" clause (ADR 0002, amended by ADR 0104).
- **Code read, not copied**: lilToon's lighting formulas [106] are re-implemented from reading its shader (MIT; ADR 0100); the Unity material converter reads Poiyomi's [122] and MToon's [130] property names; the Avatar window follows the CATS plugin's tools [83]; the physics for animation follows Cascadeur's ideas [111]. Nothing of their code is in the tree; they are credited in `CREDITS.md` and listed in `licenses/README.md` under "read, not copied".

Optional downloads are never part of a build: Blender's prebuilt libraries (fetched by the installer or by Blendity itself, each under its own licence, all GPL-3-compatible), ONNX Runtime [109] and the inbetweening network trained on the CMU motion capture database [88], and MoonRay [91] run as a separate program (ADR 0041). The Live Link bridges (`bridges/`) are Blendity's own code under the same GPL-2.0-or-later and run inside Blender, Unity, Godot and Unreal as those programs' plug-in terms allow; they link no code of Blendity's into them.

13.3 Compatible with an AI-assisted project

The code is written with Anthropic's Claude through Claude Code [85] from task specifications and decision records a person approves, and every change is reviewed by a person before it lands (`docs/AI_DISCLOSURE.md`). Three things make that compatible with the GPL and with reuse by others:

1. **The outputs are the user's.** Under Anthropic's terms [73] the model's outputs belong to the user who directed them, and the project licenses them to the extent copyright subsists in them: the human selection, direction, review and editing (a purely machine-written passage may carry no copyright of its own, and the licence claims none it does not have). Copyright is held by Emiliano Manalo and contributors; the `Co-Authored-By` trailer names the model as attribution, not as a rights holder.
2. **Nothing is claimed that isn't the project's.** Where a method comes from a paper, the paper is cited in the code's comments, in the Learn tab and in `research/FEATURES.md`; where code is copied, its licence rides along; where a program's design is followed, it is credited. A reader can tell, for every piece, what was borrowed and under what terms.
3. **The AI is a tool at build time, not in the product.** The editor runs no model except the optional, local AI inbetweening download; there is no telemetry and no cloud, and the one request it makes on its own is the optional daily check of GitHub's releases. A user of the GPL program depends on no service.

13.4 Due credit

Blendity stands on the work of others, and `CREDITS.md` names them: the Blender Foundation and the Blender authors, whose design and source every subsystem follows

[87]; Unity Technologies, whose editor the windows, shortcuts, Play mode, Animation window and asset formats follow [150]; Epic Games (Unreal: the keymap preset, the editor feel, FBX collision naming, Sequencer) [142]; the Godot Engine contributors [143]; Autodesk (Maya and 3ds Max presets) [140, 139]; Trimble (SketchUp's Push/Pull, drawing and inference) [148]; Adobe (Photoshop's brush keys and blend modes, Substance Painter's mesh maps and layer stacks, Mixamo's auto-rig) [137, 138, 84]; UModeler and Plasticity [149, 146]; Maxon's ZBrush and Pixologic's Sculptris [144, 145]; SketchBook [147], PureRef [98] and Krita [100]; VRChat [151], lilToon [106], Poiyomi [122], the VRM Consortium [130], VRoid [121], the MMD community [134] and the CATS plugin [83]; Nekki's Cascadeur [111]; DreamWorks' MoonRay [91]; REDSIM's VRC Light Volumes [124]; Carnegie Mellon's motion capture database [88]; Marschner and Shirley [69] and Gregory [67, 68], whose books the Learn tab teaches from; and the authors of every paper in the References chapter. Ideas are borrowed with their names attached; code only with its licence.

References

Everything used in Blendity, by kind: the papers whose methods became features, the books the Learn tab teaches from, the standards and licences, the libraries compiled in or linked, and the manuals of the programs Blendity follows. Each entry ends with the pages that cite it. The list is generated from the sources that cite things (`src/editor/lessons.cpp`, `research/FEATURES.md`, `licenses/README.md`) and a hand-written list (`docs/book/refs_extra.bib`); `tools/make_books.py --check` fails when an entry is never cited or a citation has no entry, so the list and the books cannot drift apart.

Papers, theses and talks

[1] R. Abdrashitov, K. Raichstat, J. Monteiro, C. Stoll, M. Dvoroznak, and A. Jacobson. “Robust Skin Weights Transfer via Weight Inpainting”. In: *SIGGRAPH Asia 2023 Technical Communications*. Cited by Learn: Modifiers vs Components. 2023 (cited on pp. 21, 53, 54).

[2] A. Akyürek and Y. Sahillioğlu. “Spring-Decomposed Skinning”. In: *Computers & Graphics* (2025). The soft-tissue bone’s lag behind the body’s acceleration, at its simplest (ADR 0049). (cited on pp. 21, 53, 54).

[3] J. Amanatides and A. Woo. “A fast voxel traversal algorithm for ray tracing”. In: *Eurographics ’87*. Cited by Learn: References. 1987 (cited on pp. 13, 53).

[4] W. Baxter, J. Wendt, and M. C. Lin. “IMPaSTo: A Realistic, Interactive Model for Paint.” In: *NPAR 2004*. Cited by research/FEATURES.md. 2004 (cited on pp. 16, 21, 50, 53).

[5] J. Brooke. “SUS: a quick and dirty usability scale.” In: *In Usability Evaluation in Industry*. Cited by research/FEATURES.md. 1996 (cited on pp. 16, 21, 51, 53).

[6] D. C. Brown. “Decentering Distortion of Lenses”. In: *Photogrammetric Engineering* 32.3 (1966). The Brown-Conrady radial model of the Lens Distortion effect. (cited on p. 14).

[7] S. K. Card, T. P. Moran, and A. Newell. “The Keystroke-Level Model for User Performance Time with Interactive Systems.” In: *CACM 23(7)* (1980). Cited by research/FEATURES.md. (cited on pp. 16, 21, 50, 53, 55).

[8] E. Catmull and J. Clark. “Recursively generated B-spline surfaces on arbitrary topological meshes.” In: *Computer-Aided Design 10(6)* (1978). Cited by Learn: Modifiers vs Components, Learn: References, research/FEATURES.md. (cited on pp. 10, 21, 48, 53).

[9] T. Chen, Z. Zheng, D. Cohen-Or, and N. J. Mitra. “3-Sweep: Extracting Editable Objects from a Single Photo.” In: *SIGGRAPH Asia 2013*. Cited by research/FEATURES.md. 2013 (cited on pp. 16, 21, 49, 53, 54).

[10] R. Cook and K. Torrance. “A reflectance model for computer graphics”. In: *ACM Transactions on Graphics 1(1)* (1982). Cited by Learn: References. (cited on pp. 12, 21).

[11] R. Cranley and T. Patterson. “Randomization of number theoretic methods for multiple integration”. In: *SIAM Journal on Numerical Analysis 13(6)* (1976). Cited by Learn: References. (cited on pp. 12, 21).

[12] C. Dachsbacher and M. Stamminger. “Reflective Shadow Maps”. In: *I3D ’05*. The sun’s reflective shadow map that voxel GI’s one bounce reads. 2005 (cited on pp. 13, 53).

[13] H. Dammertz, D. Sewtz, J. Hanika, and H. Lensch. “Edge-avoiding À-Trous wavelet transform for fast global illumination filtering”. In: *HPG ’10*. Cited by Learn: References. 2010 (cited on pp. 13, 21).

[14] G. Daviet. “Interactive Hair Simulation on the GPU Using ADMM”. In: *ACM Transactions on Graphics* 42.4 (2023). Strand contacts in a uniform grid, as the chains’ tips are pushed apart. (cited on pp. 21, 53, 54).

[15] T. Duff et al. “Building an orthonormal basis, revisited”. In: *Journal of Computer Graphics Techniques 6(1)* (2017). Cited by Learn: References. (cited on pp. 12, 21).

[16] P. M. Fitts. “The information capacity of the human motor system in controlling the amplitude of movement.” In: *J. Exp. Psychol. 47(6)* (1954). Cited by research/FEATURES.md. (cited on pp. 16, 21, 48, 53).

[17] S. F. F. Gibson. “Constrained Elastic Surface Nets: Generating Smooth Surfaces from Binary Segmented Data.” In: *MICCAI ’98*. Cited by research/FEATURES.md. 1998 (cited on pp. 16, 21, 49, 53, 54).

[18] Y. Gryaditskaya, F. Hähnlein, C. Liu, A. Sheffer, and A. Bousseau. “Lifting Freehand Concept Sketches into 3D.” In: *SIGGRAPH Asia 2020; Hähnlein, F., Gryaditskaya, Y., Sheffer, A. & Bousseau, A. (2022). *Symmetry-driven 3D Reconstruction from Concept Sketches.* SIGGRAPH 2022*. Cited by research/FEATURES.md. 2020 (cited on pp. 16, 21, 49, 53, 54).

[19] J. M. Hammersley. “Monte Carlo Methods for Solving Multivariable Problems”. In: *Annals of the New York Academy of Sciences* 86 (1960). The interleaved Hammersley ray sets of the voxel GI gather. (cited on p. 13).

[20] E. Heitz. “Sampling the GGX distribution of visible normals”. In: *Journal of Computer Graphics Techniques 7(4)* (2018). Cited by Learn: References. (cited on pp. 12, 21, 53).

[21] E. Heitz, S. Hill, and M. McGuire. “Combining analytic direct illumination and stochastic shadows”. In: *I3D ’18*. Cited by Learn: References. 2018 (cited on pp. 12, 21, 53).

[22] S. Hill. “ACES Filmic Tone Mapping Curve (an analytic fit of the RRT + ODT)”. In: *Published fit, used by Unity’s Post Processing Stack*. The “ACES (full fit, Unity’s)” view transform of round 405 is this curve without the fit’s matrices. 2016 (cited on p. 13).

[23] S. Hill, S. McAuley, B. Burley, D. Chan, Y. Gotanda, N. Hoffman, B. Smits, Y. Tokuyoshi, et al. “Physically Based Shading in Theory and Practice”. In: *SIGGRAPH 2012 course*. The split-sum environment BRDF approximation of image-based lighting (“env_brdf_approx”). 2012 (cited on p. 12).

[24] L. Hosek and A. Wilkie. “An analytic model for full spectral sky-dome radiance”. In: *SIGGRAPH ’12*. Cited by Learn: Light, Sky and Tone Mapping, Learn: References. 2012 (cited on pp. 13, 21).

[25] J. Huang, Y. Zhou, M. Niessner, J. R. Shewchuk, and L. J. Guibas. “QuadriFlow: A Scalable and Robust Method for Quadrangulation.” In: *SGP 2018*. Cited by research/FEATURES.md. 2018 (cited on pp. 16, 21, 49, 53, 54).

[26] T. Igarashi, S. Matsuoka, S. Kawachiya, and H. Tanaka. “Interactive Beautification: A Technique for Rapid Geometric Design.” In: *UIST ’97*. Cited by research/FEATURES.md. 1997 (cited on pp. 16, 21, 49, 53, 54).

[27] A. Jacobson, L. Kavan, and O. Sorkine-Hornung. “Robust Inside-Outside Segmentation using Generalized Winding Numbers.” In: *SIGGRAPH 2013*. Cited by research/FEATURES.md. 2013 (cited on pp. 16, 21, 49, 53).

[28] T. Jakobsen. “Advanced Character Physics”. In: *Game Developers Conference*. The Verlet / constraint-projection scheme of the spring bone chains (ADR 0049). 2001 (cited on pp. 21, 53, 54).

[29] J. Jimenez. “Next Generation Post Processing in Call of Duty: Advanced Warfare”. In: *SIGGRAPH 2014 course: Advances in Real-Time Rendering in Games*. The bloom’s downsample and tent-filter upsample chain (render/camera_filter.cpp). 2014 (cited on pp. 14, 53).

[30] J. T. Kajiya and T. L. Kay. “Rendering Fur with Three Dimensional Textures”. In: *SIGGRAPH ’89*. The anisotropic highlight band across the tangent used for toon hair. 1989 (cited on p. 21).

[31] B. Kang et al. “Design of advanced color temperature control system for HDTV applications”. In: *Journal of the Korean Physical Society 41(6)* (2002). Cited by Learn: References. (cited on pp. 13, 21, 53).

[32] B. Karis. “Real shading in Unreal Engine 4”. In: *SIGGRAPH course: Physically Based Shading in Theory and Practice*. Cited by Learn: References. 2013 (cited on pp. 12, 21, 53).

[33] T. Kugelstadt and E. Schömer. “Position and Orientation Based Cosserat Rods”. In: *Symposium on Computer Animation*. The bend-twist constraint the chains hold, simplified (ADR 0049). 2016 (cited on pp. 21, 53, 54).

[34] B. Lévy, S. Petitjean, N. Ray, and J. Maillot. “Least squares conformal maps for automatic texture atlas generation”. In: *SIGGRAPH ’02*. Cited by Learn: References, Learn: UV Mapping. 2002 (cited on p. 21).

[35] W. E. Lorensen and H. E. Cline. “Marching Cubes: A High Resolution 3D Surface Construction Algorithm”. In: *SIGGRAPH ’87*. The sign-grid surface extraction DynaMesh’s surface nets stand beside (ADR 0076). 1987 (cited on p. 21).

[36] M. Macklin, M. Müller, and N. Chentanez. “XPBD: Position-Based Simulation of Compliant Constrained Dynamics”. In: *Motion in Games*. Substepping and compliance in the strand and spring solvers (ADR 0049; the XPBD rods of task 0168). 2016 (cited on pp. 21, 53, 54).

[37] M. Macklin, K. Storey, M. Lu, P. Terdiman, N. Chentanez, S. Jeschke, and M. Müller. “Small Steps in Physics Simulation”. In: *Symposium on Computer Animation* (2019). Why the chain solver takes many small substeps rather than solver iterations. (cited on pp. 21, 53, 54).

[38] G. McTaggart. “Half-Life 2 / Valve Source Shading”. In: *Game Developers Conference*. The ambient cube the probe volumes’ probes store (six directions, n-squared weights). 2004 (cited on pp. 13, 53).

[39] M. Meyer, M. Desbrun, P. Schroeder, and A. Barr. “Discrete Differential-Geometry Operators for Triangulated 2-Manifolds (the cotangent Laplacian and Voronoi areas)”. In: Cited by Learn: Modifiers vs Components. 2003 (cited on p. 21).

[40] M. Mikkelsen. “Simulation of wrinkled surfaces revisited”. Cited by Learn: References. Master’s thesis. Master’s thesis, University of Copenhagen, 2008 (cited on pp. 10, 21).

[41] T. Möller and B. Trumbore. “Fast, minimum storage ray-triangle intersection”. In: *Journal of Graphics Tools 2(1)* (1997). Cited by Learn: References. (cited on pp. 12, 21).

[42] J. Motomura. “GuiltyGearXrd’s Art Style: The X Factor Between 2D and 3D”. In: *Game Developers Conference*. Toon shading’s painted inputs, custom normals and the per-vertex outline width (ADR 0048); Camera Flatten (ADR 0093). 2015 (cited on pp. 21, 54).

[43] M. Newell. “The utilization of procedure models in digital image synthesis”. Cited by Learn: References. PhD thesis. PhD thesis, University of Utah, 1975 (cited on p. 21).

[44] J. Nielsen. “Enhancing the explanatory power of usability heuristics.” In: *CHI ’94 (the ten heuristics)*. Cited by research/FEATURES.md. 1994 (cited on pp. 16, 21, 48, 53, 56).

[45] M. Okabe and K. Ito. “Color Universal Design (CUD): How to make figures and presentations that are friendly to colorblind people.” In: Cited by research/FEATURES.md. 2008 (cited on pp. 15, 21, 50, 53).

[46] B. N. Oreshkin, F. Bocquelet, F. G. Harvey, B. Raitt, and D. Laflamme. “ProtoRes: Proto-Residual Network for Pose Authoring via Learned Inverse Kinematics.” In: *ICLR*. Cited by research/FEATURES.md. 2022 (cited on pp. 16, 21, 51, 53, 54).

[47] K. Perlin. “Improving noise”. In: *SIGGRAPH ’02*. Cited by Learn: References. 2002 (cited on p. 21).

[48] B. T. Phong. “Illumination for Computer Generated Pictures”. In: *Communications of the ACM 18(6)*. The Blinn-Phong preview spheres of the Materials window. 1975 (cited on p. 16).

[49] P. G. Polson, C. Lewis, J. Rieman, and C. Wharton. “Cognitive walkthroughs: a method for theory-based evaluation of user interfaces.” In: *IJMMS 36(5)* (1992). Cited by research/FEATURES.md. (cited on pp. 16, 21, 50, 53).

[50] R. Ramamoorthi and P. Hanrahan. “An efficient representation for irradiance environment maps”. In: *SIGGRAPH ’01*. Cited by Learn: References. 2001 (cited on pp. 12, 21).

[51] L. Ruppert, S. Herholz, and H. Lensch. “Robust fitting of parallax-aware mixtures for path guiding”. In: *ACM Transactions on Graphics 39(4), SIGGRAPH ’20* (2020). Cited by Learn: References. (cited on pp. 12, 21).

[52] C. Schlick. “An inexpensive BRDF model for physically-based rendering”. In: *Computer Graphics Forum 13(3)* (1994). Cited by Learn: References. (cited on pp. 12, 21).

[53] Sheffer, Levy, Mogilnitsky, and Bogomyakov. “ABF++: Fast and Robust Angle Based Flattening”. In: *ACM TOG*. Cited by Learn: UV Mapping. 2005 (cited on pp. 21, 54).

[54] P. Shirley and K. Chiu. “A low distortion map between disk and square”. In: *Journal of Graphics Tools 2(3)* (1997). Cited by Learn: References. (cited on pp. 12, 21).

[55] K. Shoemake. “Animating Rotation with Quaternion Curves”. In: *SIGGRAPH ’85* (1985). Quaternion interpolation in the animation clips and the retargeter’s turns. (cited on p. 21).

[56] I. E. Sutherland and G. W. Hodgman. “Reentrant Polygon Clipping”. In: *Communications of the ACM* 17.1 (1974). The polygon clipping of the overlap and Z-fighting checks and of Array onto a surface. (cited on p. 10).

[57] G. Taubin. “A signal processing approach to fair surface design.” In: *SIGGRAPH ’95*. Cited by Learn: References, research/FEATURES.md. 1995 (cited on pp. 10, 21, 48, 53).

[58] The Khronos Group. *glTF 2.0 Specification*. Cited by research/FEATURES.md. , Skins and Animations, 2021 (cited on pp. 11, 17, 21, 50, 53).

[59] S. Thiedemann, N. Henrich, T. Grosch, and S. Müller. “Voxel-based Global Illumination.” In: *I3D ’11*. Cited by Learn: Baked and Realtime Lighting, Learn: References, research/FEATURES.md. 2011 (cited on pp. 13, 21, 48, 53).

[60] W3C. *Web Content Accessibility Guidelines 2.1*. Cited by research/FEATURES.md. , 1.4.3 Contrast (Minimum) and 1.4.11 Non-text Contrast, 2018 (cited on pp. 15, 21, 51, 53, 56).

[61] I. Wald. “On fast construction of SAH-based bounding volume hierarchies”. In: *IEEE Symposium on Interactive Ray Tracing*. Cited by Learn: References. 2007 (cited on pp. 12, 21).

[62] B. Walter, S. Marschner, H. Li, and K. Torrance. “Microfacet models for refraction through rough surfaces”. In: *EGSR ’07*. Cited by Learn: References. 2007 (cited on pp. 12, 21).

[63] Wang et al. “PartUV: Part-Based UV Unwrapping of 3D Meshes”. In: *SIGGRAPH Asia*. Cited by Learn: UV Mapping. 2025 (cited on pp. 21, 54).

[64] C. Welman. “Inverse Kinematics and Geometric Constraints for Articulated Figure Manipulation”. Cited by Learn: Modifiers vs Components. Master’s thesis. MSc thesis, Simon Fraser University (CCD), 1993 (cited on p. 21).

Books

[65] S. K. Card, T. P. Moran, and A. Newell. *The Psychology of Human-Computer Interaction*. The keystroke-level model’s operators and the heuristics for placing M, as docs/hci/klm.md applies them. Lawrence Erlbaum Associates, 1983 (cited on p. 16).

[66] J. M. Carroll. *The Nurnberg Funnel: Designing Minimalist Instruction for Practical Computer Skill*. Minimalist instruction: the user guide’s task-first chapters, short reading and error-recovery pages. MIT Press, 1990 (cited on p. 56).

[67] J. Gregory. *Game Engine Architecture. Volume I: Foundations and Core Engine Systems*. 3rd ed. The Learn tab’s [GEA1]: tools and the asset pipeline, profiling, SIMD, parallelism, math, memory, the game loop. CRC Press, 2018 (cited on pp. 6, 9, 16, 30, 35, 53).

[68] J. Gregory. *Game Engine Architecture. Volume II: Graphics, Motion, and Sound*. 3rd ed. The Learn tab's [GEA2]: the rendering engine, lighting, post-processing, physics, game objects, the world editor. CRC Press, 2018 (cited on pp. 11, 12, 16, 35, 53).

[69] S. Marschner and P. Shirley. *Fundamentals of Computer Graphics*. 5th ed. The Learn tab's [FoCG]: math, viewing, rasterization, meshes, texture mapping, ray tracing, colour. CRC Press, 2021 (cited on pp. 10, 12, 16, 35, 53).

[70] J. Nielsen. *Usability Engineering*. The heuristic evaluation method behind the HCI passes (ADR 0085). Academic Press, 1993 (cited on pp. 16, 56).

Standards, specifications and licences

[71] Academy of Motion Picture Arts and Sciences. *Academy Color Encoding System (ACES)*. The RRT + ODT that the ACES view transforms approximate. AMPAS, 2026. URL: https://acescentral.com/ (cited on p. 13).

[72] American Society of Cinematographers. *ASC Color Decision List (CDL) Transfer Functions and Interchange Syntax*. The lift / gain / gamma order of the Color Grading effect. ASC Technology Committee, 2009 (cited on p. 14).

[73] Anthropic. *Anthropic Consumer Terms of Service and Commercial Terms*. The terms under which the model's outputs belong to the user who directed them (the authorship statement). Anthropic, 2026. URL: https://www.anthropic.com/legal/consumer-terms (cited on pp. 34, 56).

[74] Blender Foundation. *FBX binary file format specification (the io_scene_fbx exporter's encode_bin.py)*. The record tree, header, null records and footer the FBX writer follows. Blender Foundation, 2026. URL: https://code.blender.org/2013/08/fbx-binary-file-format-specification/ (cited on pp. 11, 18).

[75] Free Software Foundation. *GNU General Public License, version 2*. The licence of Blendity's source (GPL-2.0-or-later), as Blender's. Free Software Foundation, 1991. URL: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html (cited on pp. 33, 56).

[76] Free Software Foundation. *GNU General Public License, version 3*. The licence of the executables, which combine GPL-2.0-or-later code with Apache-2.0 libraries. Free Software Foundation, 2007. URL: https://www.gnu.org/licenses/gpl-3.0.html (cited on pp. 33, 56).

[77] Free Software Foundation. *Frequently Asked Questions about the GNU Licenses*. Why Apache-2.0 code combines with GPL-3 and not GPL-2-only (the "or later" clause). Free Software Foundation, 2026. URL: https://www.gnu.org/licenses/gpl-faq.html (cited on p. 33).

[78] ISO/TC 171. *ISO 14289-2:2024, Document management applications - PDF/UA-2: Accessibility of PDF*. The accessible-PDF profile the books aim at: tags, language, alt text, a logical reading order. International Organization for Standardization, 2024. URL: https://www.iso.org/standard/83216.html (cited on p. 56).

[79] M. Korth. *PSX-SPX: PlayStation Specifications (GPU Rendering, Dither Matrix)*. The Retro Console filter's PS1 look: integer vertices, affine texture mapping, the 4 x 4 dither matrix. no$psx, 2024. URL: https://psx-spx.consoledev.net/ (cited on p. 14).

[80] Pixar Animation Studios. *Universal Scene Description (USD) Documentation*. The USDA text form Blendity reads and writes. Pixar, 2026. URL: https://openusd.org/ (cited on pp. 11, 18).

[81] The Apache Software Foundation. *Apache License, Version 2.0*. The licence of Embree, OpenImageDenoise, oneTBB, MikkTSpace and the Cycles sky code ported in src/render/sky.cpp. The Apache Software Foundation, 2004. URL: https://www.apache.org/licenses/LICENSE-2.0 (cited on pp. 33, 56).

[82] The Linux Foundation. *SPDX License List and Identifiers*. The 'SPDX-License-Identifier' headers in every source file. SPDX Workgroup, 2024. URL: https://spdx.org/licenses/ (cited on p. 33).

Software and libraries

[83] absolute-quantum. *CATS Blender Plugin*. MIT. Fix Model, Create Visemes, Decimation, Eye Tracking, Merge Armatures: the Avatar window follows its tools (rounds 356-361). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on pp. 34, 35, 53).

[84] Adobe. *Adobe Mixamo*. The marker auto-rig's workflow (chin, wrists, elbows, knees, groin) and the bone naming the humanoid map knows. 2026. URL: https://www.mixamo.com/ (cited on p. 35).

[85] Anthropic. *Claude Code*. The agentic development environment Blendity is written with, under human direction and review (docs/AI_DISCLOSURE.md). 2026. URL: https://claude.com/claude-code (cited on pp. 22, 34, 56).

[86] AppImage project. *AppImage and appimagetool*. The Linux package (packaging/linux/make_packages.sh). 2026. URL: https://appimage.org/ (cited on p. 27).

[87] Blender Foundation and the Blender authors. *Blender*. Version 5.2. GPL-2.0-or-later. The design and source Blendity's meshes, Edit Mode tools, UVs, materials, renderers, skins and IK follow; the Learn lessons name the file each piece mirrors. 2026. URL: https://projects.blender.org/blender/blender (cited on pp. 1, 3, 10, 11, 33, 35, 56).

[88] Carnegie Mellon University. *CMU Graphics Lab Motion Capture Database*. The data the AI inbetweening network and Puppet Posing's pose prior were trained on: "The data used in this project was obtained from mocap.cs.cmu.edu. The database was created with funding from NSF EIA-0196217." 2026. URL: http://mocap.cs.cmu.edu/ (cited on pp. 34, 35, 53).

[89] *cuew*. Comp. software. loads NVIDIA's CUDA driver at run time (builds with OptiX: the Windows and Linux installers since task 0169). Licence: Modified Apache 2.0. Cited by licenses/README.md. 2026 (cited on pp. 51, 53, 54).

[90] *DPC++ runtime*. Comp. software. Version 7.1.0. runtime dependency of Embree's build (SYCL). Licence: Apache-2.0 with LLVM exception. Cited by licenses/README.md. 2026. URL: https://github.com/intel/llvm (cited on pp. 52, 53).

[91] DreamWorks Animation. *OpenMoonRay*. Apache-2.0. An optional external renderer run as a program of its own (ADR 0041, moonray.yml); not linked, not shipped. 2026. URL: https://github.com/dreamworksanimation/openmoonray (cited on pp. 19, 34, 35).

[92] *Eigen*. Comp. software. Version (commit 8a1083e). LSCM sparse solver (header-only). Licence: MPL-2.0. Cited by licenses/README.md. 2026. URL: http://eigen.tuxfamily.org (cited on pp. 51, 53, 54).

[93] *Embree*. Comp. software. Version 4.4.1. path tracer ray casting. Licence: Apache-2.0. Cited by licenses/README.md. 2026. URL: https://github.com/RenderKit/embree (cited on pp. 51, 53, 54).

[94] *fast_float*. Comp. software. Version 5.0.0. fast, exact float parsing in the scene loader. Licence: MIT. Cited by licenses/README.md. 2026. URL: https://github.com/fastfloat/fast_float (cited on p. 51).

[95] *FFmpeg*. Comp. software. Version 8.1. Render Animation's MP4, encoded in-process (task 0077). Licence: LGPL-2.1-or-later ; Blender's build enables GPL parts (x264, x265), so as built it is GPL-2.0-or-later. Cited by licenses/README.md. 2026. URL: https://ffmpeg.org/ (cited on pp. 52–55).

[96] GitHub. *GitHub Actions*. The build, release and MoonRay workflows run on it. 2026. URL: https://docs.github.com/actions (cited on p. 53).

[97] *Hosek-Wilkie sky model*. Comp. software. physical sky. Licence: BSD-3-Clause. Cited by licenses/README.md. 2026 (cited on pp. 51, 53).

[98] Idyllic Pixel. *PureRef*. The reference board's way of working (ADR 0089). 2026. URL: https://www.pureref.com/ (cited on p. 35).

[99] *Jolt Physics*. Comp. software. Version 5.6.0. Play-mode rigid bodies. Licence: MIT. Cited by licenses/README.md. 2026. URL: https://github.com/jrouwe/JoltPhysics (cited on pp. 51, 53, 54).

[100] Krita Foundation. *Krita*. GPL-3.0. The canvas-turning keys (4, 5, 6) the Draw tab answers. 2026. URL: https://krita.org/ (cited on p. 35).

[101] *LAME*. Comp. software. Version 3.100. MP3, inside FFmpeg. Licence: LGPL-2.0-or-later. Cited by licenses/README.md. 2026. URL: https://lame.sourceforge.io/ (cited on pp. 52–54).

[102] *libaom*. Comp. software. Version 3.13.1. AV1, inside FFmpeg (aom.dll on Windows). Licence: BSD-2-Clause (and the AOM patent licence). Cited by licenses/README.md. 2026. URL: https://aomedia.googlesource.com/aom/ (cited on pp. 52–54).

[103] *libjpeg-turbo*. Comp. software. Version 2.1.3. JPEG decoding. Licence: BSD-3-Clause (and the IJG terms in its notices). Cited by licenses/README.md. 2026. URL: https://github.com/libjpeg-turbo/libjpeg-turbo/ (cited on pp. 51, 53).

[104] *libpng*. Comp. software. Version 1.6.58. PNG decoding. Licence: libpng-2.0. Cited by licenses/README.md. 2026. URL: http://www.libpng.org/pub/png/libpng.html (cited on pp. 51, 53).

[105] *libvpx, Opus, libvorbis / libogg / libtheora*. Comp. software. Version 1.15.2, 1.3.1, -. codecs inside FFmpeg. Licence: BSD-3-Clause. Cited by licenses/README.md. 2026. URL: https://github.com/webmproject/libvpx (cited on p. 52).

[106] lilxyzw. *lilToon*. MIT. Its lighting model's formulas (lil_common_frag.hlsl) are re-implemented in src/render/shading.cpp (ADR 0100); no code is copied. 2026. URL: https://github.com/lilxyzw/lilToon (cited on pp. 34, 35).

[107] *Manifold*. Comp. software. Version 3.5.2. Boolean modifier. Licence: Apache-2.0. Cited by licenses/README.md. 2026. URL: https://github.com/elalish/manifold (cited on pp. 51, 53, 54).

[108] *meshoptimizer*. Comp. software. Version 1.1. Decimate modifier. Licence: MIT. Cited by licenses/README.md. 2026. URL: https://meshoptimizer.org (cited on pp. 51, 53).

[109] Microsoft. *ONNX Runtime*. Version 1.31.0. MIT. The optional download that runs the inbetweening network (ADR 0046); only its C API's headers are in the tree. 2026. URL: https://onnxruntime.ai/ (cited on p. 34).

[110] *MikkTSpace*. Comp. software. tangents for normal maps. Licence: Apache-2.0. Cited by licenses/README.md. 2026 (cited on pp. 51, 53, 54).

[111] Nekki. *Cascadeur*. The ideas behind physics for animation: the centre of mass, contacts, ballistic flight, AutoPhysics and AutoPosing (ADRs 0042-0045, 0087). Ideas only; no code. 2026. URL: https://cascadeur.com/ (cited on pp. 34, 35).

[112] *NVIDIA OptiX's loader*. Comp. software. compiled into builds with OptiX's headers - the Windows and Linux installers since task 0169 (ADR 0054; ADR 0037 before it). Licence: BSD-3-Clause. Cited by licenses/README.md. 2026 (cited on pp. 51, 53).

[113] *oneTBB*. Comp. software. Version 2022.3.0. job system. Licence: Apache-2.0. Cited by licenses/README.md. 2026. URL: https://github.com/uxlfoundation/oneTBB (cited on pp. 51, 53).

[114] *Open PGL*. Comp. software. Version 0.7.1. path guiding. Licence: Apache-2.0. Cited by licenses/README.md. 2026. URL: http://www.openpgl.org/ (cited on pp. 51, 53).

[115] *OpenColorIO*. Comp. software. Version 2.5.0. AgX / Filmic / other view transforms. Licence: BSD-3-Clause. Cited by licenses/README.md. 2026. URL: https://github.com/AcademySoftwareFoundation/OpenColorIO (cited on pp. 51, 53–55).

[116] *OpenEXR + Imath*. Comp. software. Version 3.4.10 / 3.2.2. EXR read/write. Licence: BSD-3-Clause. Cited by licenses/README.md. 2026. URL: https://github.com/AcademySoftwareFoundation/openexr (cited on pp. 51, 53).

[117] *OpenImageDenoise*. Comp. software. Version 2.5.0. path tracer denoising. Licence: Apache-2.0. Cited by licenses/README.md. 2026. URL: https://www.openimagedenoise.org/ (cited on pp. 51, 53, 54).

[118] *OpenJPH*. Comp. software. Version 0.25.2. runtime dependency of OpenEXR. Licence: BSD-2-Clause. Cited by licenses/README.md. 2026. URL: https://github.com/aous72/OpenJPH (cited on pp. 51, 53).

[119] *OpenSubdiv*. Comp. software. Version 3.7.0. optional Catmull-Clark backend. Licence: TOST-1.0, Apache-2.0 based. Cited by licenses/README.md. 2026. URL: https://graphics.pixar.com/opensubdiv (cited on pp. 51, 53, 54).

[120] *OpenXR SDK*. Comp. software. Version 1.1.65. VR View (round 383, ADR 0096): the loader, loaded at run time, reaches the headset's OpenXR runtime. Licence: Apache-2.0 OR MIT (,). Cited by licenses/README.md. 2026. URL: https://github.com/KhronosGroup/OpenXR-SDK-Source (cited on pp. 52–54).

[121] pixiv Inc. *VRoid Studio*. The VRM avatars and the J_Bip bone names Blendity's imports and the humanoid map read. 2026. URL: https://vroid.com/en/studio (cited on p. 35).

[122] Poiyomi. *Poiyomi Toon Shader*. Read for its material properties (the Unity material converter and the Poiyomi texture preset). 2026. URL: https://github.com/poiyomi/PoiyomiToonShader (cited on pp. 34, 35, 53).

[123] *QuadriFlow*. Comp. software. Version 27a6867. quad remeshing (task 0241); with it LEMON 1.3.1 (Boost Software License 1.0) and pcg32 (Apache-2.0); pss (Intel, BSD-3-Clause) is in the copy but not compiled. Licence: BSD-3-Clause (, the text of - Blender's README.blender says MIT, the file is BSD-3;). Cited by licenses/README.md. 2026. URL: https://github.com/hjwdzh/QuadriFlow (cited on pp. 51, 53–55).

[124] REDSIM. *VRC Light Volumes*. The Light Volume component and its baking (rounds 397-398, ADR 0097); the package stays in the user's Unity project. 2026. URL: https://github.com/REDSIM/VRCLightVolumes (cited on pp. 35, 53, 54).

[125] J. Russell. *Inno Setup*. The Windows installer (packaging/windows/blendity.iss). 2026. URL: https://jrsoftware.org/isinfo.php (cited on p. 27).

[126] *shaderc*. Comp. software. Version v2025.4. compiles the GPU path-tracing kernel (GLSL to SPIR-V) at run time. Licence: Apache-2.0 (glslang also BSD-3-Clause / MIT parts). Cited by licenses/README.md. 2026. URL: https://github.com/google/shaderc (cited on pp. 52–54).

[127] The Khronos Group. *OpenXR*. The VR View's runtime API (ADR 0096). 2026. URL: https://www.khronos.org/openxr/ (cited on p. 16).

[128] *The OptiX kernel's PTX*. Comp. software. Blendity's own GPL kernel compiled with NVRTC against NVIDIA's OptiX device headers (their _optix_* intrinsic calls), committed (task 0169, ADR 0054). Licence: GPL-2.0-or-later (Blendity's), the device headers' calls NVIDIA's. Cited by licenses/README.md. 2026 (cited on pp. 51, 53).

[129] *ufbx*. Comp. software. Version v0.20.0. FBX import. Licence: MIT. Cited by licenses/README.md. 2026. URL: https://github.com/ufbx/ufbx (cited on pp. 51, 53).

[130] VRM Consortium. *UniVRM / MToon*. MIT. The VRM 0.x / 1.0 glTF extension and the MToon material Blendity's VRM import reads. 2026. URL: https://github.com/vrm-c/UniVRM (cited on pp. 18, 34, 35, 53).

[131] *Vulkan-Headers*. Comp. software. Version 1.4.341. GPU rendering API declarations (headers only; the Vulkan loader comes with the GPU driver). Licence: Apache-2.0 / MIT. Cited by licenses/README.md. 2026. URL: https://github.com/KhronosGroup/Vulkan-Headers (cited on pp. 52–54).

[132] *x264*. Comp. software. Version 35fe20d. H.264 encoding, inside FFmpeg. Licence: GPL-2.0-or-later. Cited by licenses/README.md. 2026. URL: https://www.videolan.org/developers/x264.html (cited on pp. 52–54).

[133] *x265*. Comp. software. Version 4.1. H.265 encoding, inside FFmpeg (Linux, macOS). Licence: GPL-2.0-or-later. Cited by licenses/README.md. 2026. URL: https://www.videolan.org/developers/x265.html (cited on pp. 52–54).

[134] Yu Higuchi and the MMD community. *MikuMikuDance (PMX, PMD and VMD file formats)*. The .pmx / .pmd model and .vmd motion files Blendity imports (ADR 0047). 2026. URL: https://sites.google.com/view/vpvp/ (cited on pp. 18, 35).

[135] *zlib*. Comp. software. Version 1.3.1. PNG compression. Licence: Zlib. Cited by licenses/README.md. 2026. URL: https://zlib.net (cited on pp. 52–54).

[136] *Zstandard*. Comp. software. Version 1.5.7. compressed .scene files. Licence: BSD-3-Clause. Cited by licenses/README.md. 2026. URL: https://github.com/facebook/zstd (cited on pp. 52, 53).

Manuals and websites

[137] Adobe. *Adobe Photoshop User Guide*. The Adobe keymap preset (B, E, V, [ ]), the 27 blend modes, Free Transform and the Layers window Blendity follows. 2026. URL: https://helpx.adobe.com/photoshop/user-guide.html (cited on p. 35).

[138] Adobe. *Adobe Substance 3D Painter Documentation*. Mesh maps, the layer stack, smart masks and smart materials (ADRs 0079-0082). 2026. URL: https://helpx.adobe.com/substance-3d-painter/home.html (cited on p. 35).

[139] Autodesk. *Autodesk 3ds Max Documentation*. The 3ds Max keymap preset (Chamfer, Cut, Clone). 2026. URL: https://help.autodesk.com/view/3DSMAX/ (cited on p. 35).

[140] Autodesk. *Autodesk Maya Documentation*. The Maya keymap preset (Q W E R, F8-F11, Set Key on S) and the Maya editor feel. 2026. URL: https://help.autodesk.com/view/MAYAUL/ (cited on p. 35).

[141] *CATS Blender Plugin: Create Visemes*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 53).

[142] Epic Games. *Unreal Engine Documentation*. The Unreal keymap preset and editor feel, FBX collision naming, Sequencer, Live Link. 2026. URL: https://dev.epicgames.com/documentation/ (cited on pp. 19, 35).

[143] Godot Engine contributors. *Godot Engine Documentation*. The TSCN format, importing 3D scenes, WorldEnvironment; the Godot bridge. 2026. URL: https://docs.godotengine.org/ (cited on pp. 18, 35).

[144] Maxon. *ZBrush Documentation*. Sculpt Mode's brushes, masks, polygroups, DynaMesh and ZRemesher (ADRs 0074-0077). 2026. URL: https://help.maxon.net/zbrush/ (cited on p. 35).

[145] Pixologic. *Sculptris (dynamic tessellation)*. Dyntopo / Sculptris Pro: the mesh tessellated under the brush (ADR 0078). 2026. URL: https://pixologic.com/sculptris/ (cited on p. 35).

[146] Plasticity. *Plasticity Documentation*. Construction planes and lines, the perpendicular snap, Shell, Thicken, Draft, Radial Array, Fillet. 2026. URL: https://plasticity.xyz/ (cited on p. 35).

[147] Sketchbook, Inc. *SketchBook Pro Documentation*. The rulers, ellipse, French curves, symmetry and perspective guides, the toolbar's order, the brush puck (ADRs 0063, 0088). 2026. URL: https://www.sketchbook.com/ (cited on p. 35).

[148] Trimble. *SketchUp Help Center*. Push/Pull's semantics (ADR 0004), the drawing tools, inference snapping, Follow Me, the SketchUp keymap preset. 2026. URL: https://help.sketchup.com/ (cited on p. 35).

[149] UModeler, Inc. *UModeler X Documentation*. Drawing shapes onto faces, Follow, Lathe, Slice, the Shift-drag extrude. 2026. URL: https://umodeler.gitbook.io/ (cited on p. 35).

[150] Unity Technologies. *Unity User Manual (Unity 6)*. The editor's windows, shortcuts, Play mode, lighting, the Animation window, Humanoid avatars and the YAML asset formats Blendity follows. 2026. URL: https://docs.unity3d.com/Manual/ (cited on pp. 1, 35).

[151] VRChat Inc. *VRChat Creator Documentation*. The Avatar Descriptor, PhysBones, the performance ranks, the Worlds SDK; the Avatar window and the Profiler's Budget follow them. 2026. URL: https://creators.vrchat.com/ (cited on pp. 30, 35).

Index A: the code and its references

Every source file that implements a published method, and every library the program compiles in or links, with the reference it comes from. Generated by `tools/make_books.py --bib` from `research/FEATURES.md` and `licenses/README.md`; the page numbers after each reference in the References chapter lead back here and to the chapters that discuss it.

Source	What it implements	References
`src/research/research.cpp`	Taubin λ|μ smoothing (`taubin_smoothing`) (implemented (example))	[57]
`src/research/research.cpp`	Add Surface Noise (`surface_noise`), noisy input for comparing smoothers (utility)	(utility, no paper)
`docs/tasks/0235-hci-pass.md`, `src/editor/tab_strip.cpp`	The HCI audit of the editor's windows, tabs, menus and bars (task 0235): visibility of status (tabs never clipped, the breadcrumb), recognition over recall (tab groups, a stage's tabs), consistency (one signal per tab state), memory load (the guide hides after two sessions), help (a tool's next step in the status bar) (applied (an audit, not an algorithm))	
`docs/tasks/0235-hci-pass.md`	Reach on a 13-inch 2-in-1: related tabs folded into one with a member row (one target instead of three), the hidden tabs one button away (+N), the Inspector folded with one button (task 0235) (applied)	[16]
`src/scene/mesh.cpp`	Catmull-Clark subdivision (core Mesh tool + SubdivisionSurface component) (implemented (core))	[8]
`src/render/voxel_gi.cpp`	Realtime GI (Lighting window > Realtime GI): binary voxel columns, the mip-map ray test (sec. 4), one bounce of the sun through a reflective shadow map and sky occlusion (sec. 5.1) in the rasterized views, with baked lightmaps (task 0013, ADR 0011) (implemented (renderer, CPU))	[59]

Source	What it implements	References
`src/editor/sketch_build.cpp`	Solids from a few strokes: a box from three lines at a corner, a cylinder or cone from two circles, a sweep from two circles and a path (`sketchbuild`); here on the sketch's scene canvases instead of a photograph, the solids as F9 operators (ADR 0065) (implemented (task 0225))	[9]
`src/research/sketch_lift.cpp`	A scaffold of construction lines lifted from the picture plane: shared junctions, lines along the axes' vanishing directions, depths by least squares (`sketchlift`, ADR 0067); the symmetry and curved lines of the papers not taken (experimental, gate met (task 0227))	[18]
`src/scene/sketch.cpp`	Pen strokes corrected at the lift: split at corners, fitted to lines, arcs, ellipses and curves, pulled to the guides (ADR 0063) (implemented (task 0223))	[26]
`src/scene/dynamesh.cpp`	DynaMesh's inside test: the winding number along grid lines of three axes, the nonzero rule merging self-intersections, a majority of three votes a grid vertex (ADR 0076) (implemented (the grid-line form, not the paper's hierarchical evaluation))	[27]
`src/scene/dynamesh.cpp`	DynaMesh's surface: naive surface nets over the sign grid (a vertex per mixed cell at the mean of its edge crossings, a quad per mixed grid edge), then tangential relaxation (implemented)	[17]
`src/scene/quad_remesh.cpp`	Quad Remesh: Blender's patched QuadriFlow compiled in (`extern/quadriflow`), driven as Blender's wrapper does, on a worker with a cancel flag (ADR 0077) (integrated (library))	[25]
`src/scene/dyntopo.cpp`	Dyntopo: edges under the brush longer than the detail size split at their midpoints, shorter ones collapsed where that folds no triangle (ADR 0078) (implemented)	(no paper: Sculptris / Blender's Dyntopo)

Source	What it implements	References
`src/render/mesh_maps.cpp`	Mesh maps baked into the UV layout: world normal, position in the bounds, a sagitta-over-chord curvature smoothed twice, cosine-weighted occlusion and thickness through the path tracer on the mesh alone (ADR 0079) (implemented)	(no paper: Substance Painter's mesh maps)
`src/texture/surface.cpp`, `src/texture/smart_material.cpp`	The surface's stack with mask stacks reading the mesh maps, the rect engine redoing only the edited layers inside the edited rectangle, smart materials as saved groups, texture sets packed per engine (ADRs 0080, 0082) (implemented)	(no paper: Substance Painter's layer stack, smart masks and smart materials)
`src/editor/paint.cpp`	Painting by projection: the brush's disc projected through the camera, texels placed by barycentrics and depth-tested against the view's planes; a stroke one undo step of the 64 x 64 tiles it touched (ADR 0081) (implemented)	(no paper: Blender's Project Paint)
`src/editor/paint.cpp`	The painterly brush's wet mixing: the brush carries the mean colour it went over and mixes the next dab by a wet amount - a one-value pickup, not IMPaSTo's paint volume and Kubelka-Munk mixing (ADR 0083) (simplified)	[4]
`src/scene/stroke_object.cpp`	Stroke objects' Canopy normals: each card's custom normal points away from a centre so a bush of flat cards shades as one round volume (ADR 0084) (implemented)	(no paper: the hand-painted foliage trick)
`docs/hci/klm.md`, `tools/hci_klm.py`	Nineteen frequent actions counted before and after the HCI pass (K, P, B, H, M with the paper's rules for M), recounted by a tool (ADR 0085) (applied (a model, not an algorithm))	[7]
`docs/hci/walkthrough-*.md`	Four pipelines walked at the baseline and after the pass, the four questions per step (ADR 0085) (applied)	[49]
`src/editor/sculpt.cpp`	Sculpt polygroups in the eight colours, then lighter and darker tiers (task 0269) (implemented)	[45]
`src/scene/export.cpp`	Skinned meshes exported with joints, inverse bind matrices, JOINTS_0 / WEIGHTS_0 and sampled channels (task 0268) (implemented)	[58]

Source	What it implements	References
`tools/posemodel/train.py`, `src/scene/puppet.cpp`	Puppet Posing's pose prior: any subset of the ends' goals to the whole pose, a residual network trained with goals hidden (task 0280) (implemented (much smaller, positions only))	[46]
`src/editor/ui.cpp`	The relative-luminance contrast ratio (`ui::contrast_ratio`) measuring every theme; the High Contrast theme at 4.5:1 or more (task 0261) (implemented)	[60]
`docs/hci/study-protocol.md`, `docs/hci/study-282.md`	The study's questionnaire (ADR 0085) (protocol written, **not run**)	[5]
ufbx v0.20.0	FBX import	[129]
fast_float 5.0.0	fast, exact float parsing in the scene loader	[94]
MikkTSpace	tangents for normal maps	[110]
Hosek-Wilkie sky model	physical sky	[97]
QuadriFlow 27a6867	quad remeshing (task 0241); with it LEMON 1.3.1 (Boost Software License 1.0) and pcg32 (Apache-2.0); pss (Intel, BSD-3-Clause) is in the copy but not compiled	[123]
cuew	loads NVIDIA's CUDA driver at run time (builds with OptiX: the Windows and Linux installers since task 0169)	[89]
NVIDIA OptiX's loader	compiled into builds with OptiX's headers - the Windows and Linux installers since task 0169 (ADR 0054; ADR 0037 before it)	[112]
The OptiX kernel's PTX	Blendity's own GPL kernel compiled with NVRTC against NVIDIA's OptiX device headers (their `_optix_*` intrinsic calls), committed (task 0169, ADR 0054)	[128]
Embree 4.4.1	path tracer ray casting	[93]
OpenImageDenoise 2.5.0	path tracer denoising	[117]
oneTBB 2022.3.0	job system	[113]
Eigen (commit 8a1083e)	LSCM sparse solver (header-only)	[92]
OpenSubdiv 3.7.0	optional Catmull-Clark backend	[119]
OpenEXR + Imath 3.4.10 / 3.2.2	EXR read/write	[116]
OpenJPH 0.25.2	runtime dependency of OpenEXR	[118]
OpenColorIO 2.5.0	AgX / Filmic / other view transforms	[115]
Manifold 3.5.2	Boolean modifier	[107]
meshoptimizer 1.1	Decimate modifier	[108]
Open PGL 0.7.1	path guiding	[114]
Jolt Physics 5.6.0	Play-mode rigid bodies	[99]
libjpeg-turbo 2.1.3	JPEG decoding	[103]
libpng 1.6.58	PNG decoding	[104]

Source	What it implements	References
zlib 1.3.1	PNG compression	[135]
Zstandard 1.5.7	compressed `.scene` files	[136]
DPC++ runtime 7.1.0	runtime dependency of Embree's build (SYCL)	[90]
shaderc v2025.4	compiles the GPU path-tracing kernel (GLSL to SPIR-V) at run time	[126]
Vulkan-Headers 1.4.341	GPU rendering API declarations (headers only; the Vulkan loader comes with the GPU driver)	[131]
OpenXR SDK 1.1.65	VR View (round 383, ADR 0096): the loader, loaded at run time, reaches the headset's OpenXR runtime	[120]
FFmpeg 8.1	Render Animation's MP4, encoded in-process (task 0077)	[95]
x264 35fe20d	H.264 encoding, inside FFmpeg	[132]
x265 4.1	H.265 encoding, inside FFmpeg (Linux, macOS)	[133]
libaom 3.13.1	AV1, inside FFmpeg (`aom.dll` on Windows)	[102]
libvpx, Opus, libvorbis / libogg / libtheora 1.15.2, 1.3.1, -	codecs inside FFmpeg	[105]
LAME 3.100	MP3, inside FFmpeg	[101]

Index B: the documents and their references

The project's documents (the README, the architecture, the manual, the decision records, the HCI reports) and the references each one names. Generated by `tools/make_books.py --bib`: a document is matched to a reference when it names the first author and the year on one line, or the reference's title.

Document	References it draws on
README.md	[96, 67, 68, 69, 92, 93, 95, 107, 115, 117, 114, 119, 123, 104, 113, 126, 135]
docs/ARCHITECTURE.md	[2, 3, 83, 141, 12, 14, 21, 20, 25, 28, 29, 31, 32, 33, 36, 37, 38, 46, 59, 124, 92, 93, 95, 99, 107, 110, 115, 116, 117, 118, 114, 119, 123, 131, 136, 89, 103, 104, 108, 113, 126, 129, 135]
docs/MANUAL.md	[1, 83, 141, 59, 124, 93, 95, 99, 107, 115, 117, 120, 123, 108]
docs/LIVELINK.md	[115]
docs/PERFORMANCE.md	[92, 93, 107, 110, 115, 117, 114, 119, 123, 136, 103, 104, 108, 113, 126, 135]
docs/USABILITY.md	[124, 93, 95, 117, 123]
research/FEATURES.md	[4, 5, 7, 8, 9, 16, 17, 18, 25, 26, 27, 58, 44, 45, 46, 49, 57, 59, 60, 123]
licenses/README.md	[83, 88, 130, 122, 90, 92, 93, 95, 97, 99, 101, 107, 110, 112, 115, 116, 117, 118, 114, 119, 120, 123, 128, 131, 136, 89, 102, 103, 104, 108, 113, 126, 129, 132, 133, 135]
CREDITS.md	[1, 83, 96, 67, 68, 130, 69, 124, 110, 123, 89, 129]
docs/decisions/0001-optional-blender-libraries.md	[92, 93, 107, 115, 119, 104, 108, 126, 135]
docs/decisions/0002-gpl-licensing.md	[110, 129]
docs/decisions/0004-sketchup-push-pull.md	[107]
docs/decisions/0011-voxel-gi.md	[59]
docs/decisions/0013-gpu-viewport.md	[126]
docs/decisions/0014-gi-on-the-gpu.md	[126]
docs/decisions/0021-gpu-background-open.md	[126]
docs/decisions/0022-deform-only-path.md	[110]
docs/decisions/0025-bones-and-skins.md	[129]

Document	References it draws on
docs/decisions/0026-minimum-spec-and-quality-tiers.md	[93, 117, 119]
docs/decisions/0028-inpainted-skin-weights.md	[1, 92, 107]
docs/decisions/0032-ffmpeg.md	[95, 101, 102, 132, 133, 135]
docs/decisions/0033-updates-and-library-fallback.md	[95]
docs/decisions/0034-render-mesh-in-place.md	[107, 110]
docs/decisions/0036-views-on-the-gpu.md	[115]
docs/decisions/0037-cuda-optix-build-time-sdks.md	[89]
docs/decisions/0041-moonray-external-renderer.md	[95]
docs/decisions/0043-autophysics-centroidal-qp.md	[92]
docs/decisions/0044-active-ragdoll-jolt.md	[99]
docs/decisions/0046-ai-inbetweening.md	[46]
docs/decisions/0048-anime-rendering.md	[42, 126]
docs/decisions/0049-hair-and-soft-tissue-physics.md	[2, 14, 28, 33, 36, 37]
docs/decisions/0054-shipping-optix.md	[89]
docs/decisions/0057-part-based-uv-unwrapping.md	[53, 63, 92, 107]
docs/decisions/0061-native-gpu-views.md	[131]
docs/decisions/0063-sketch-canvases-and-guides.md	[26]
docs/decisions/0065-solids-from-construction-lines.md	[9]
docs/decisions/0067-lifting-a-scaffold.md	[18]
docs/decisions/0075-dynamic-subdivision-by-local-re-evaluation.md	[119]
docs/decisions/0076-dynamesh-by-winding-columns-and-surface-nets.md	[17, 107]
docs/decisions/0077-zremesher-spike-quadriflow.md	[25, 92, 107, 123]
docs/decisions/0078-sculptris-pro-dynamic-tessellation.md	[107]
docs/decisions/0096-vr-openxr.md	[120]
docs/decisions/0097-vrc-light-volumes.md	[124]

Document	References it draws on
docs/decisions/0099-unity-scene-look.md	[115]
docs/hci/checklist.md	[95]
docs/hci/compare-2026-10.md	[95, 123]
docs/hci/klm.md	[7]
docs/hci/review-313.md	[95]

Colophon

Authorship and copyright. Blendity is copyright Emiliano Manalo and contributors. Its source code, tests, documentation and these two books were written with Anthropic's Claude through Claude Code [85], under a human's direction and review: the user decides what Blendity is for, approves each task's specification and decision record before code is written, reviews every pull request and merges it. The model's outputs belong to the user who directed them under Anthropic's terms [73], and are licensed to the extent copyright subsists in them (the human selection, direction, review and editing); the `Co-Authored-By` trailer on every commit is attribution, not a licence grant. No third-party code is claimed as the project's own: what is copied is listed in `licenses/README.md` with its licence, what is followed is credited in `CREDITS.md` and the References chapter, and the whole story is in `docs/AI_DISCLOSURE.md`.

Licence. The source is GPL-2.0-or-later [75], the same as Blender's [87]; the executables, which combine it with Apache-2.0 libraries [81], are distributed under GPL-3.0-or-later [76]. The scenes, meshes, textures, renders and animations made with Blendity are their maker's: the GPL covers the program, not its output.

How the books are made. The sources are LaTeX under `docs/book/`, typeset with LuaLaTeX in the DejaVu faces (the editor's own) and biblatex with biber for the reference list. `tools/make_books.py` generates the reference list and the index tables from the project's own files, takes the figures headless from a scratch project, builds both PDFs and checks that nothing has drifted. The release workflow builds them again on every tag and attaches them to the release.

Accessibility and readability. The books follow the guidelines the editor itself is measured against (ADR 0085), applied to documents: Nielsen's tenth heuristic, help and documentation [44, 70] (easy to search: a table of contents, bookmarks, a list of figures, a reference list with back-references and three indexes; focused on the user's task: the user guide is the work stage by stage with numbered steps and worked examples, the architecture book the questions a maintainer asks; not too large: each chapter ends in a pointer to the Learn lesson that goes deeper); Carroll's minimalist instruction [66] (start on a real task, keep the reading short, support error recovery: a troubleshooting chapter and the "when something refuses" list); and WCAG 2.1 [60] as PDF/UA applies it [78]: every text colour, link and box title reads at 4.5:1 or more (the ratios are in `blendity-common.sty`), links are told apart by form as well as colour (bracketed citation numbers, "chapter 3", monospace URLs), every figure carries alt text and a caption that says what it shows, the diagrams' meaning is in their captions and the text around them, the document's language and metadata are declared and its headings are the PDF's bookmarks (structure tags for a screen reader are the one open item: LaTeX's tagging project does not yet support the packages the books use, so the captions carry what the alt text would), the body type is about 10 points of a face made for screens with 80 characters a line, and nothing is conveyed by colour alone. Keys, menus and controls are typeset consistently: Ctrl+K is a key, Window > Layouts a menu path, Generate Lighting a control's label, `make_books.py` a file or command.

Version. Blendity 0.13.0, November 2026, from the `main` branch of `https://github.com/EmilianoManaloIV/blendity`.